

華中科技大學

自主智能系統

— 智能感知技術 (SLAM)

馬杰

slam概述

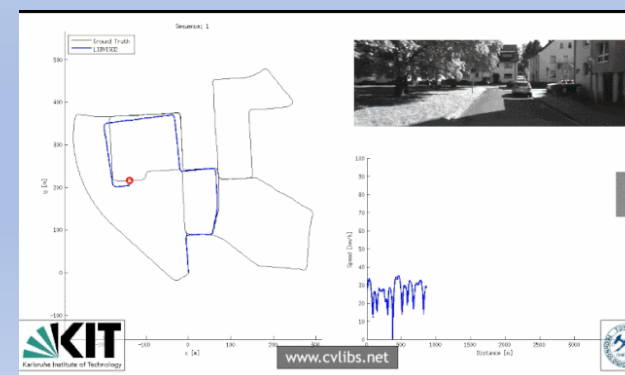
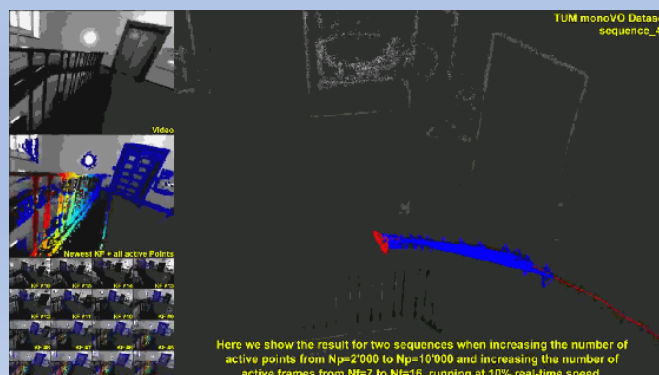
Simultaneous Localization And Mapping

- 机器人自主运动两大基本问题
 - 我在什么地方? — 定位
 - 周围长什么样子? — 建图
 - 定位侧重对自身的了解, 建图侧重对外在的了解

准确的定位
需要精确的地图

精确的地图
来自准确的定位

正是因为地图和定位之间的耦合, 带来一些列问题。。。

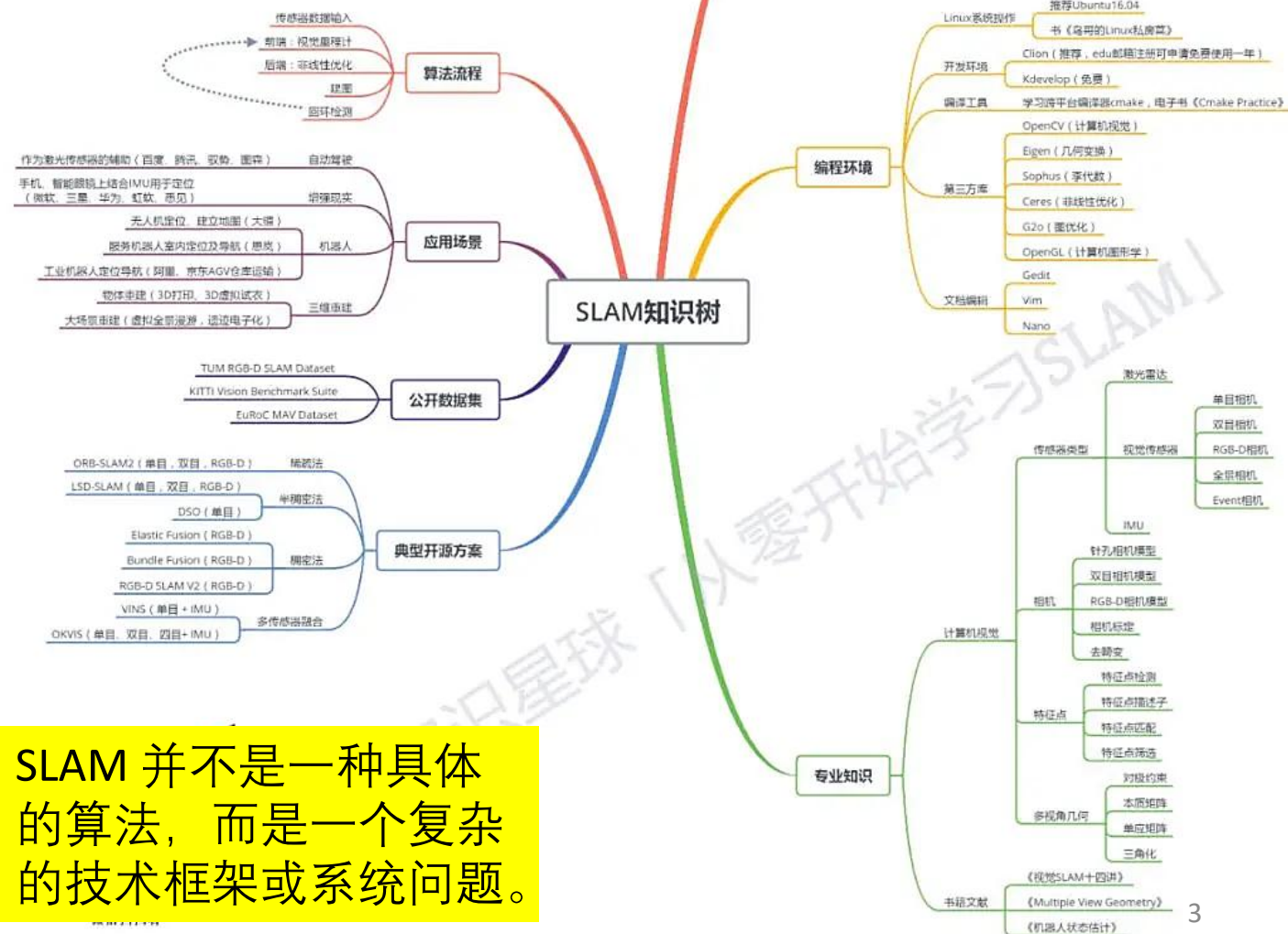
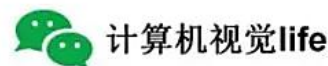


SALM知识树

SLAM 涉及：线性代数矩阵运算、李群李代数求导、三维空间刚体变换、相机成像模型、特征提取匹配、多视角几何、非线性优化、回环检测、集束调整、三维重建等专业知识。

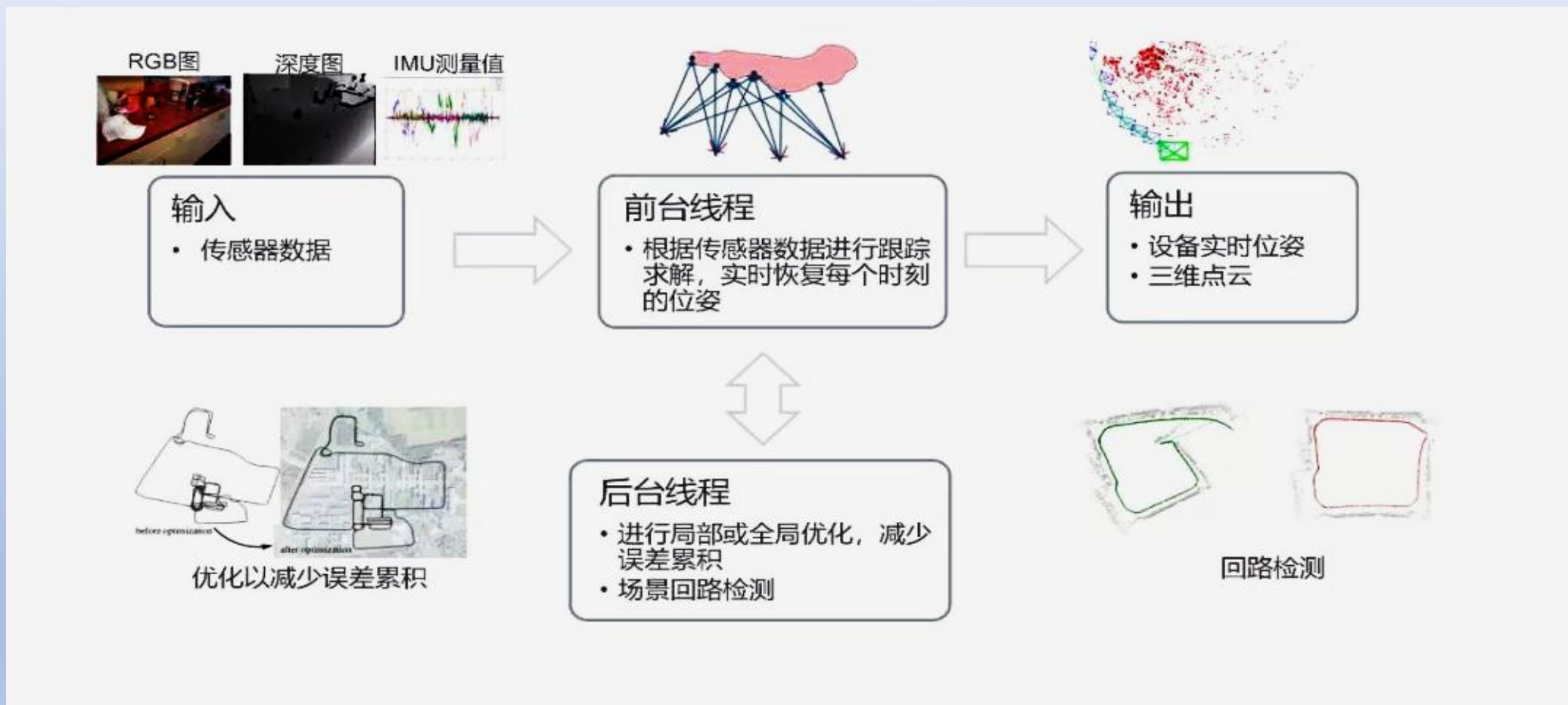


不要急着“上吊”啊。。。



SLAM 并不是一种具体的算法，而是一个复杂的技术框架或系统问题。

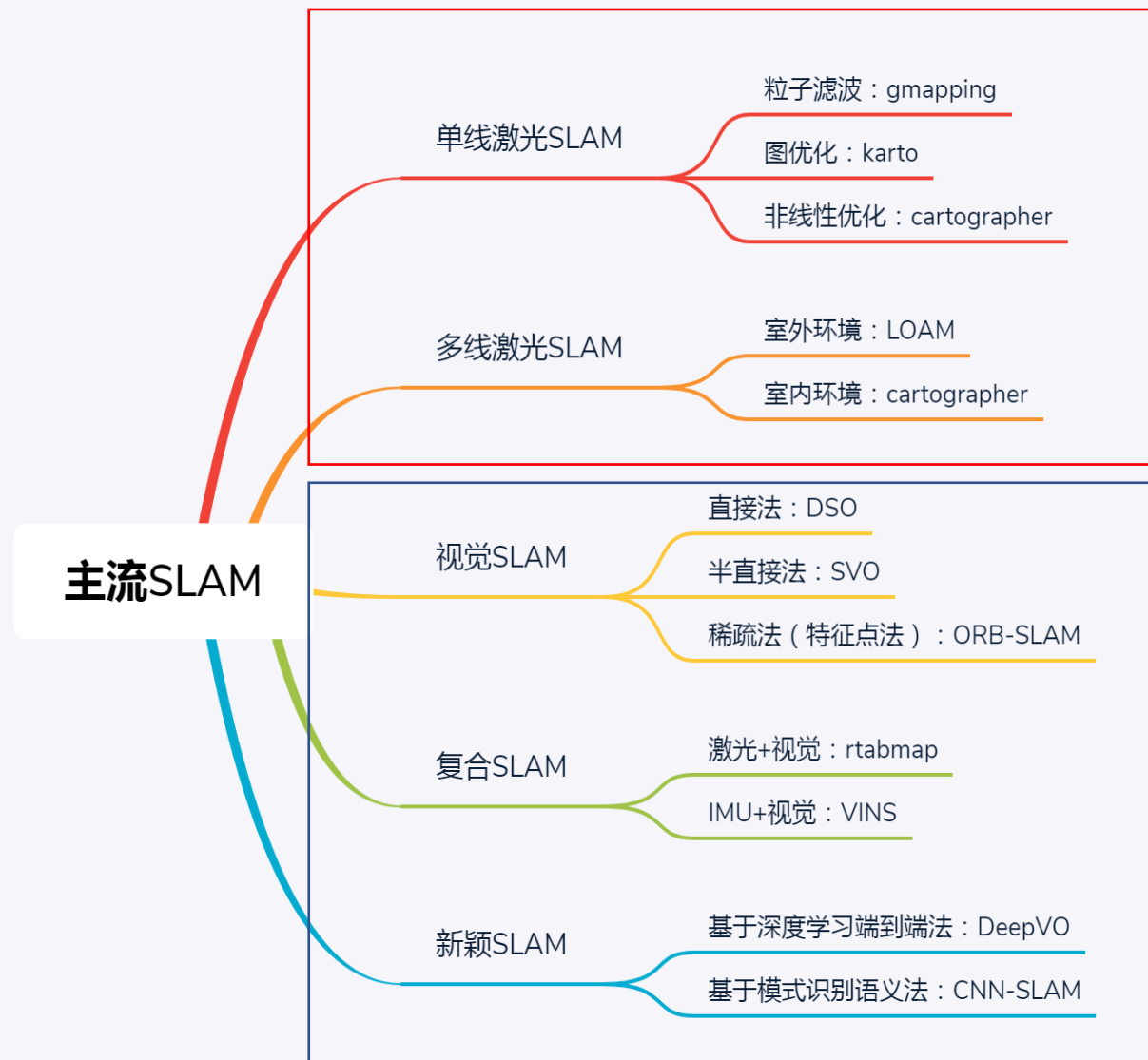
SLAM系统主流框架



SLAM技术经过多年的发展，目前主流SLAM系统采用的是2007年牛津大学提出的并行跟踪与建图框架。

SLAM分类

- 按传感器来分，主要分为两大类：
- 激光SLAM
- 视觉SLAM



激光SLAM VS 视觉SLAM

优/劣势	激光SLAM	VSLAM
优	可靠性高、技术成熟	结构简单，安装的方式多元化
	建图直观、精度高、 不存在累计误差	无传感器探测距离限制，成本低
	地图可用于路径规划	可提取语义信息
劣	受Lidar探测范围限制	环境光影响大、暗处（无纹理区域）无法工作
	安装有结构要求	运算负荷大，构建的地图本身难以直接用于路径规划与导航
	地图缺乏语义信息	传感器动态性能还需提高，地图构建时会存在累计误差

传感器

SLAM的实现方式与难度，与传感器形式和安装方式有很大关联。

激光传感器

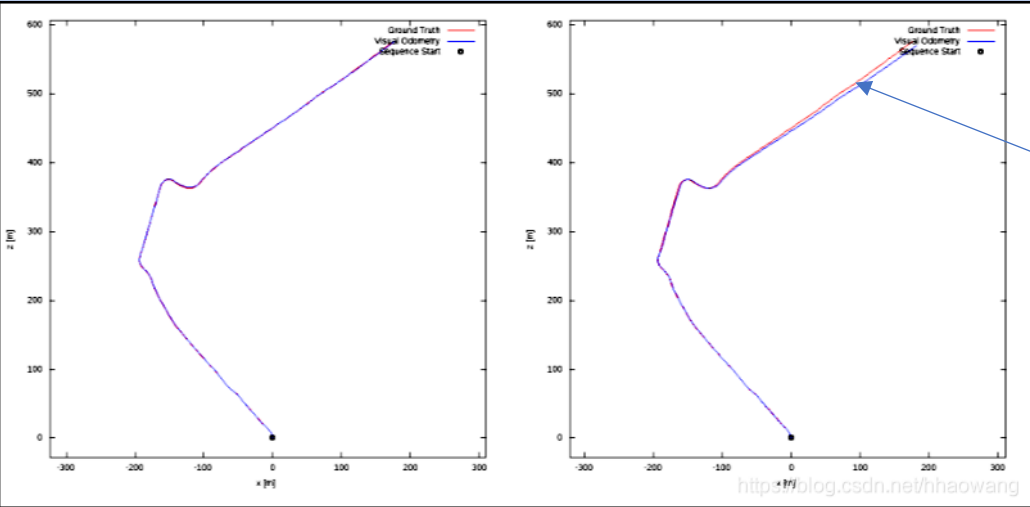
- 精确
- 快速
- 计算量小
- 研究充分
- 体积大、功耗大
- 价格昂贵

视觉传感器

- 便宜
- 轻量
- 信息丰富
- 计算量大
- 对环境假设强
- 易受干扰
- 细分：单目、双目、RGB-D



对于同一个场景，VSLAM在后半程中出现了偏差，这是因为累积误差所引起的，所以VSLAM要进行回环检验。



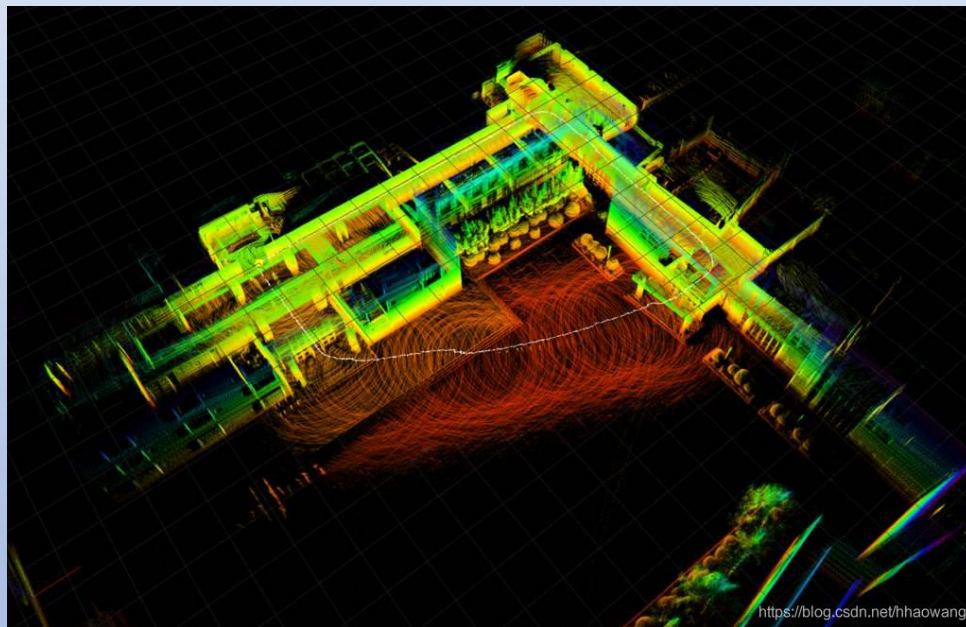
激光SLAM 视觉SLAM 典型方案

算法方案 (激光)	核心方法	定位精度	典型应用场景与特点
Gmapping	基于粒子滤波 (RBPF)	厘米级 (室内小场景)	扫地机器人、小范围室内。计算量小、精度高, 但依赖里程计, 不适用于大场景或空旷环境。
Hector SLAM	高斯-牛顿法 (图优化)	分米级 (无里程计环境下)	无人机、崎岖地面。仅依赖激光雷达, 无需里程计, 适合地面不平或空中应用。
Cartographer	图优化 + 闭环检测	厘米级 (室内外大型场景)	仓储机器人、大型建筑。谷歌开源, 通过“子图”和闭环检测消除累积误差, 是目前工业界应用最广的2D方案之一。
LeGO-LOAM	特征提取 (地面/分割) + ICP	厘米级	地面无人车。轻量化版本, 优化了地面特征提取, 在计算资源有限时表现良好。
LIO-SAM	紧耦合因子图优化	厘米级 (高精度)	工业巡检、自动驾驶。目前最主流的高精度方案之一, 通过紧耦合将激光、IMU等多源数据融合, 在剧烈振动或高速运动下依然稳健。
CT-ICP	连续时间ICP优化	相对平移误差 < 0.59%	自动驾驶、高速移动。专为高动态、高速运动场景设计, 能有效校正运动畸变。
PIN-SLAM	隐式表示 + 全局优化	相对平移误差 < 0.51%	前沿研究、复杂环境。结合了神经隐式表示等新技术, 是目前精度非常高的算法之一。

算法方案 (视觉)	核心方法	定位精度	典型应用场景与特点
ORB-SLAM3	基于特征点 (ORB) + 图优化 + 多地图复用	厘米级 (室内外通用)	最成熟通用的方案。支持单目、双目、RGB-D相机, 集成了IMU紧耦合, 具有重定位和多地图功能, 是目前工业界和学术界应用最广的方案。
VINS-Mono / VINS-Fusion	紧耦合 (滑窗优化) + 预积分	厘米级	视觉-惯性融合标杆。专为无人机、手持设备等高速运动场景设计, 计算效率高, 支持回环检测和全局优化。VINS-Fusion支持多传感器 (双目/GPS) 融合。
SVO 2.0	半直接法 (特征点 + 直接法) + 深度滤波器	分米级 (高帧率)	高速/轻量级场景。追求计算效率, 适用于无人机、AR/VR等对实时性要求极高但精度要求相对宽松的场景。2.0版本加入了回环和重定位。
DSO (Direct Sparse Odometry)	直接法 (光度误差) + 稀疏优化	厘米级 (纹理丰富环境)	无特征依赖。在弱纹理、运动模糊场景下比ORB-SLAM更鲁棒, 但无全局回环, 适合对漂移不敏感的前端里程计任务。
LSD-SLAM	直接法 + 半稠密深度估计	分米级	大尺度场景。能够构建半稠密3D地图, 适合室外大范围建图, 但现已逐渐被DSO替代。
基于深度学习的方案 (如 DROID-SLAM, TartanVO)	可微优化 / 端到端学习	精度提升 (强泛化)	前沿研究方向。利用神经网络替代传统模块 (特征提取、匹配、优化), 在动态环境、极端光照下表现更鲁棒, 但对算力要求高。

激光SLAM

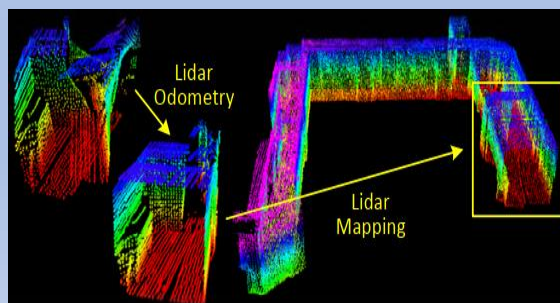
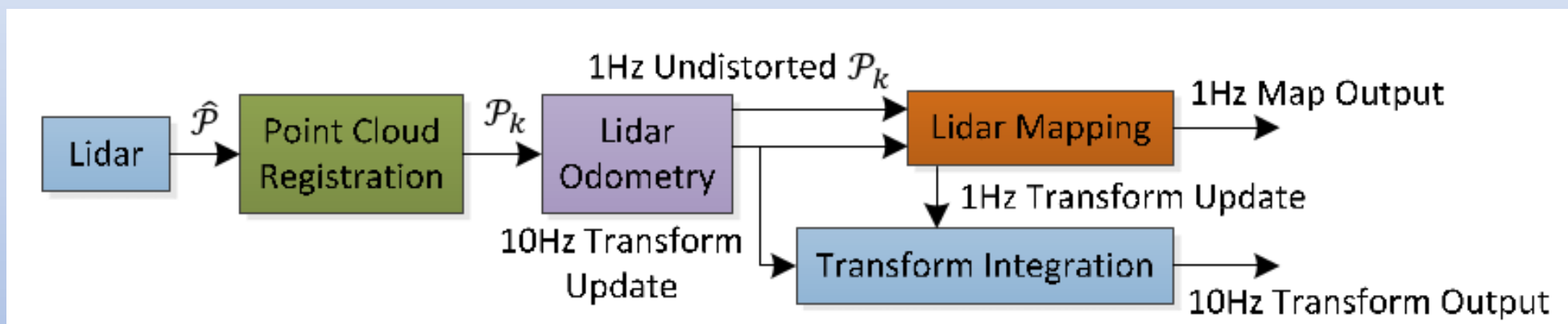
在2005年的时候，激光SLAM就已经被研究的比较透彻，框架也已初步确定。



主动发射激光，依靠回波信息，直接测量距离、方位、速度等信息
和VSLAM的优势：3D估计3D，精度高

LOAM 方案

LOAM (Lidar Odometry and Mapping) 是2014年提出的一项里程碑式的激光雷达SLAM算法。它最核心的思想是将复杂的定位与建图问题，巧妙地“解耦”成两个并行的线程：一个高频但相对粗略，另一个低频但十分精确。很好地平衡了算法的实时性和精度。

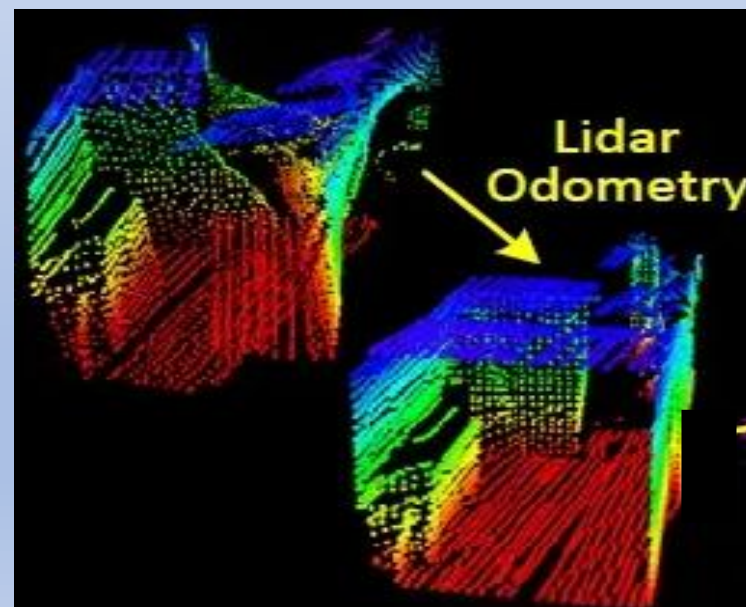
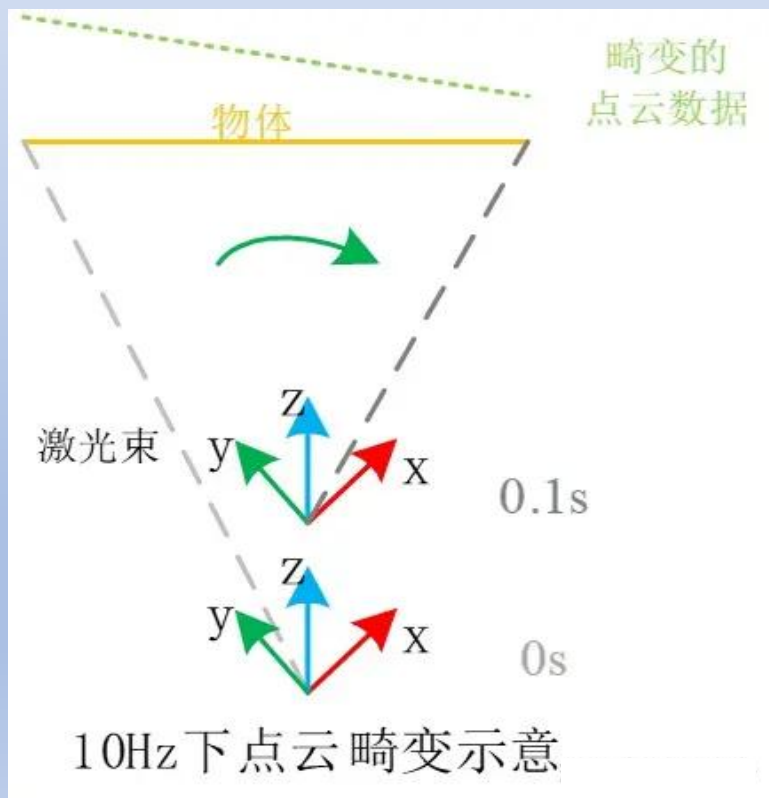


点云注册:构造匀速模型，去除不可靠点，提取点云特征。
激光里程计:帧点云配准，计算里程计信息。10Hz
三维建图:局部点云融合到全局地图，计算精确位姿。1Hz
坐标发布:发布坐标信息，供导航定位使用。

点云注册 - Pointcloud Registration

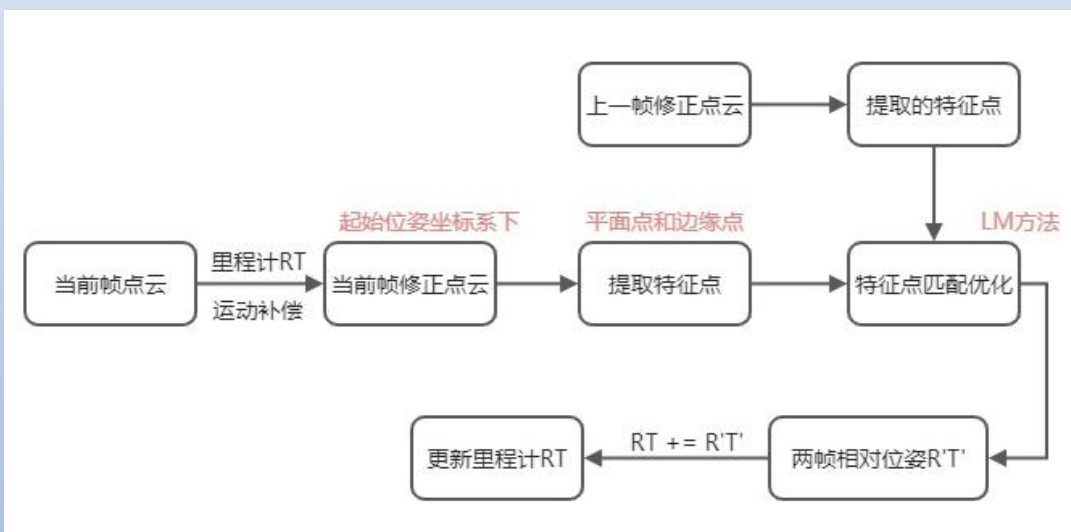
构造**匀速**模型，去除**不可靠**点，提取点云**特征**。

产生原因：扫描激光雷达带来的运动畸变。
解决方法：雷达转速、自身运动速度已知，**按照匀速模型线性补偿**。



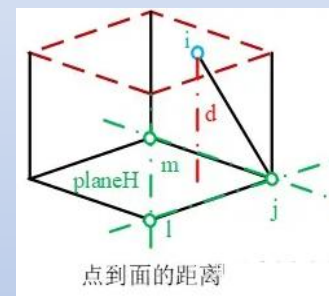
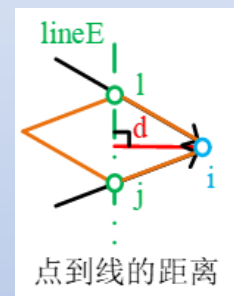
激光里程计 - Odometry

目标：帧-帧点云配准，计算运动方向和距离。@10Hz
特点：不处理所有的激光点，而是筛选出两类最具代表性的特征点，大大降低了计算量



1. 通过计算连续几个点的曲率，提取**边缘点**和**平面点**；
2. 分别计算两帧边缘点和平面点的距离差（**点到线**、**点到面的距离**）；
3. 所有点的**距离残差d**作为优化目标函数
4. 获得最优的**R和T**（例如高斯牛顿法）。

$$c = \frac{1}{|S| \cdot \|X_{k,i}^L\|} \left\| \sum_{j \in S, j \neq i} (X_{k,i}^L - X_{k,j}^L) \right\|$$

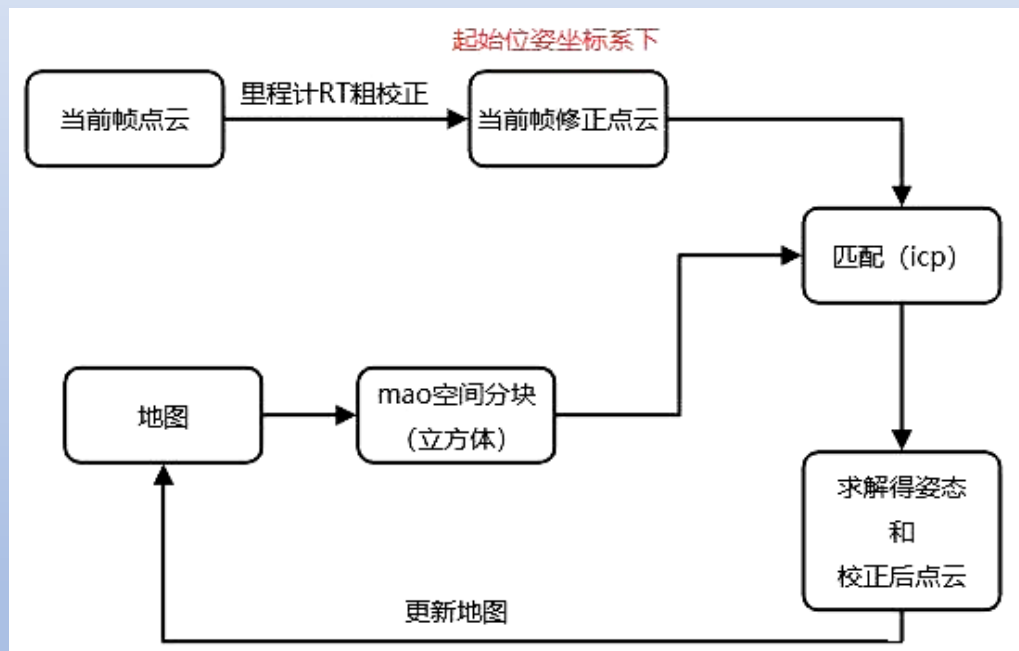


$$f(T_{k,k+1}^L) = \begin{bmatrix} f_E(X_{k,a_1}^L, T_{k,a_1}^L) \\ f_E(X_{k,a_2}^L, T_{k,a_2}^L) \\ \dots \\ f_H(X_{k,a_1}^L, T_{k,a_1}^L) \\ f_H(X_{k,a_2}^L, T_{k,a_2}^L) \end{bmatrix} = \begin{bmatrix} d_{E,a_1} \\ d_{E,a_2} \\ \dots \\ d_{H,a_1} \\ d_{H,a_2} \end{bmatrix} = d$$

$$J^T J \Delta T_{k,k+1}^L = -J^T d$$

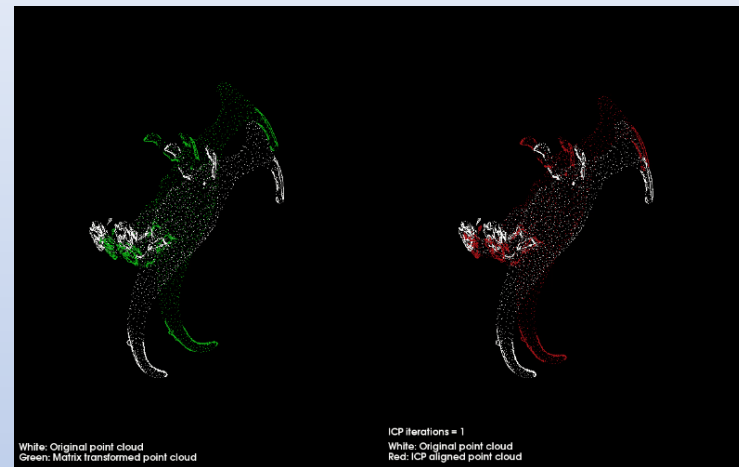
制图-Mapping

计算当前帧相对于地图的**精确**位姿，把**局部**点云融合到**全局**地图。
@1Hz

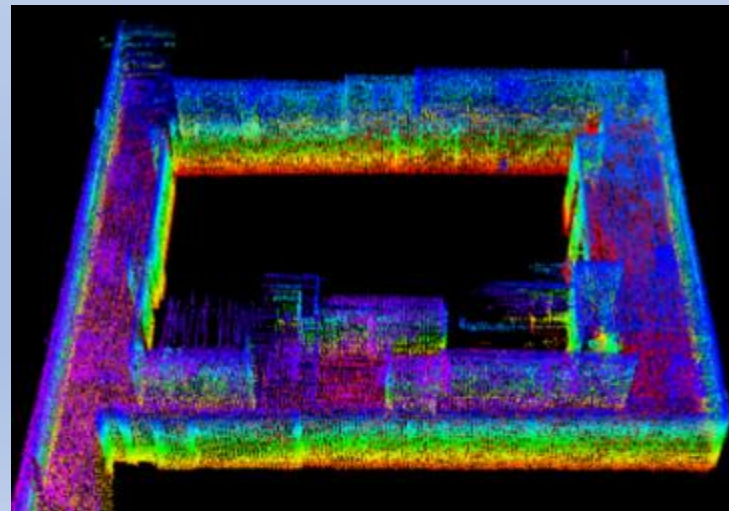


和Odometry流程相比：

- 1) 特征提取的方法和之前一样,但是点数增加10倍
- 2) 匹配空间扩大到10米*10米的立方体
- 3) 采取更精细的匹配方法（例如ICP）



ICP迭代最近点算法

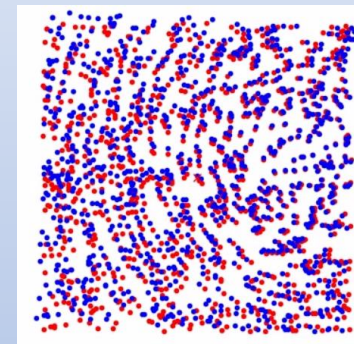


ICP算法

- 目前应用最广泛的点云精配准算法，迭代最近点算法（Iterative Closest Point, ），存在各种变种 ICP 算法。

$$R^*, t^* = \arg \min_{R, t} \frac{1}{|P_s|} \sum_{i=1}^{|P_s|} \|p_t^i - (R \cdot p_s^i + t)\|^2$$

其中 p_s^i 和 p_t^i 是原点云 P_s (source)和目标点云 P_t (target) 中的对应点。



ICP 把点云配准问题拆分成了两个子问题：

- 1) 找最近点，计算总体误差
- 2) 找最优变换，调整RT。
- 3) 循环迭代1-2。一般终止条件有：
 - a. RT变化量小于阈值
 - b. loss变化量小于阈值
 - c. 达到最大迭代次数

ICP 优点：

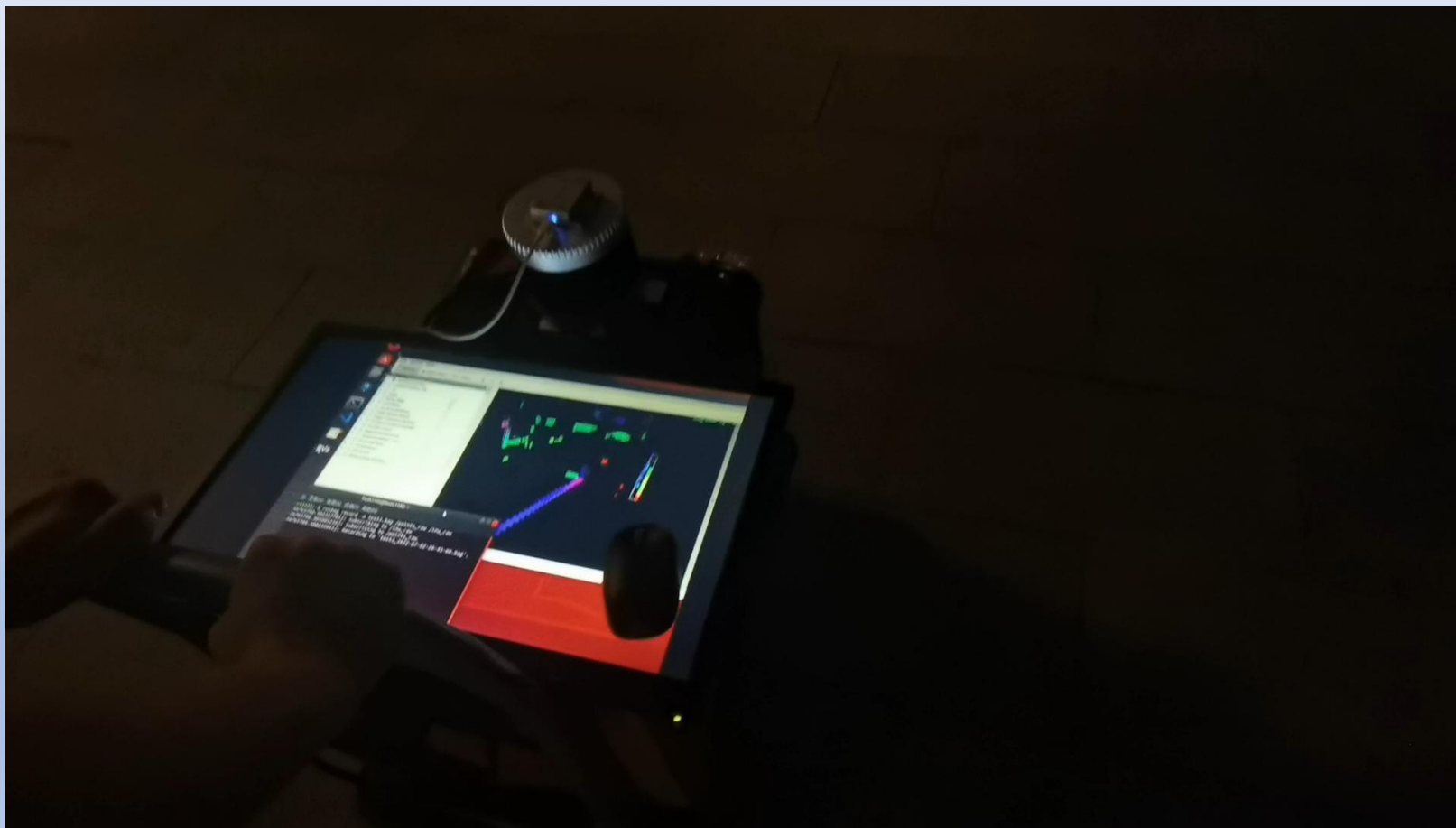
- 简单，不必对点云进行分割和特征提取
- 初值较好情况下，精度和收敛性都不错

ICP 缺点：

- 找最近对应点的计算开销较大
- 初值误差不能太大

只考虑点与点距离，缺少对点云结构信息的利用

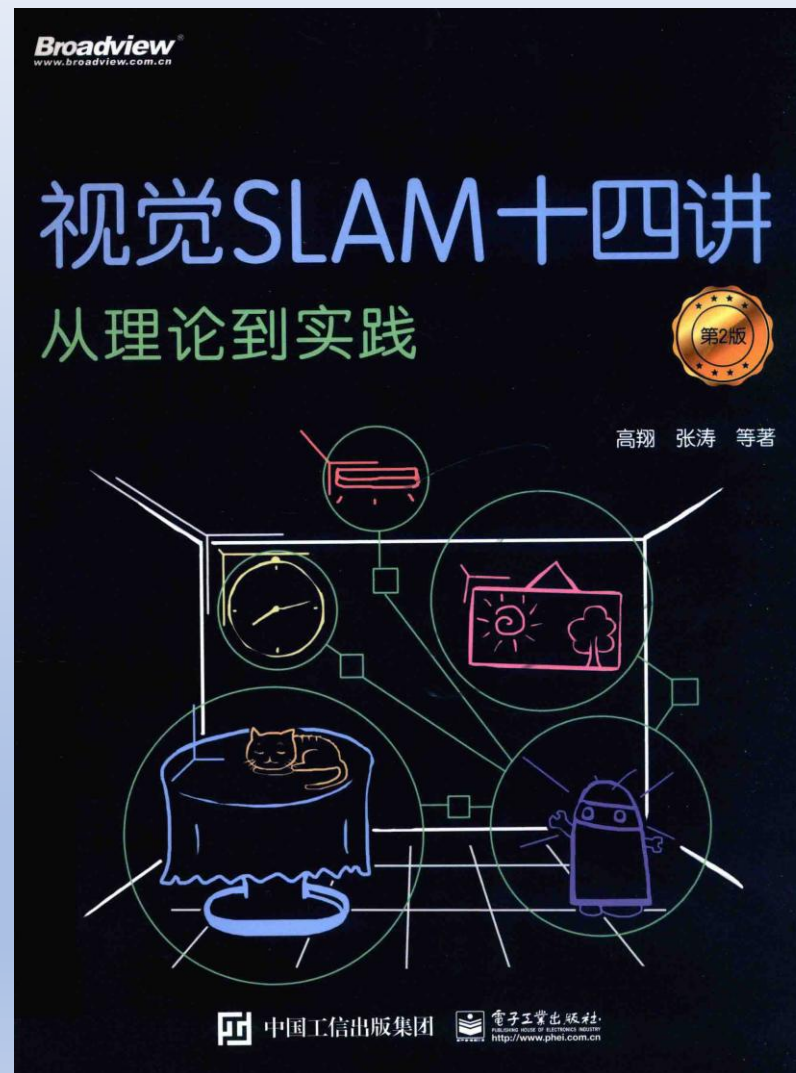
激光SLAM实例



视觉SALM技术

1. SLAM概述
2. 相机与图像
3. 视觉里程计
4. 后端优化
5. 回环检测
6. 地图构建

本课程 14in1



VSLAM概述

• 分类

- 单目 Monocular: 通过移动相机产生深度
- 双目 Stereo: 通过视差计算深度
- 深度 RGBD: 通过物理方法测量深度
- 其他 全景 Event Camera, etc.



单目	双目	RGB-D
成本低	计算深度	主动测深度
距离不受限	距离不受限	重建效果好
尺度不确定性	配置复杂	测量范围小
初始化问题	计算量大	受日光干扰
		受材质干扰



单目视觉SLAM

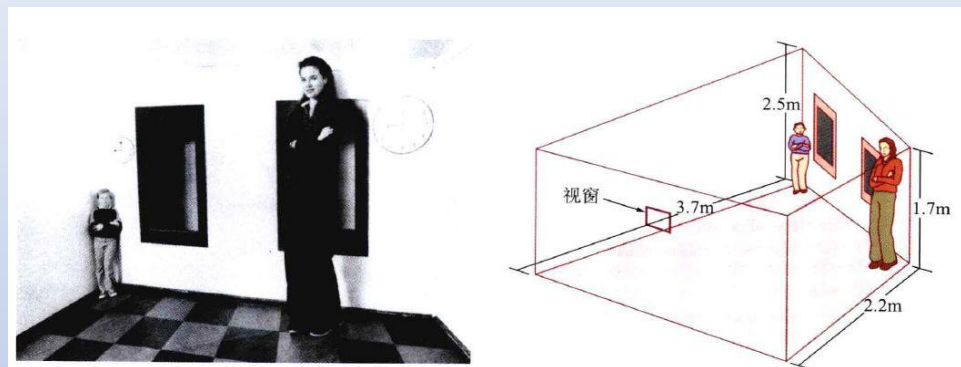
仅有一个图像时无法判断大小和距离：

- 可能是很近但很小的物体
- 可能是很远但很大的物体

必须在移动相机后才能得知相机的运动和场景的结构： **Structure from motion**

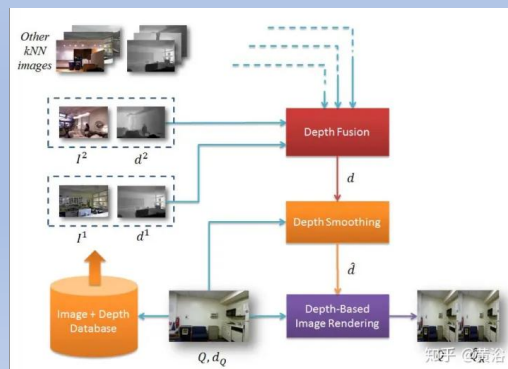
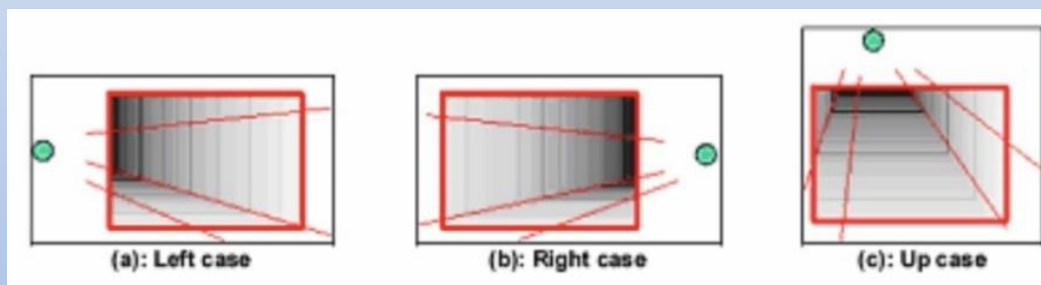
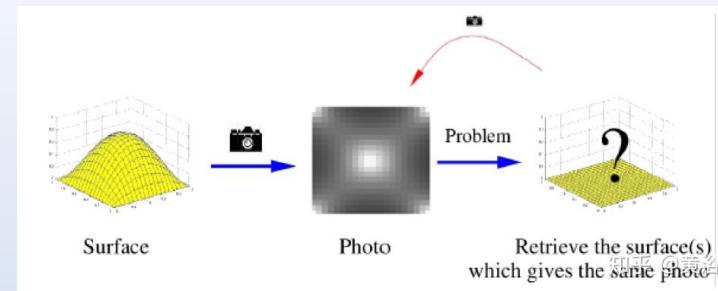
当相机运动起来时，场景和成像有几何关系：

近处物体的像运动快
远处物体的像运动慢
可以推断距离

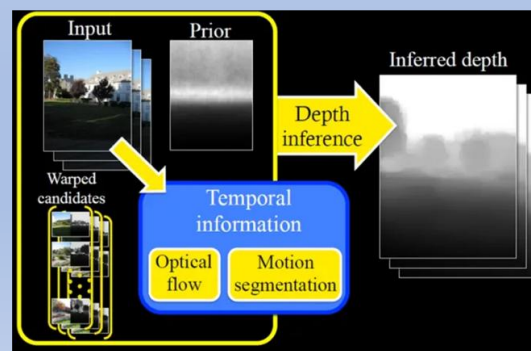


单目深度估计

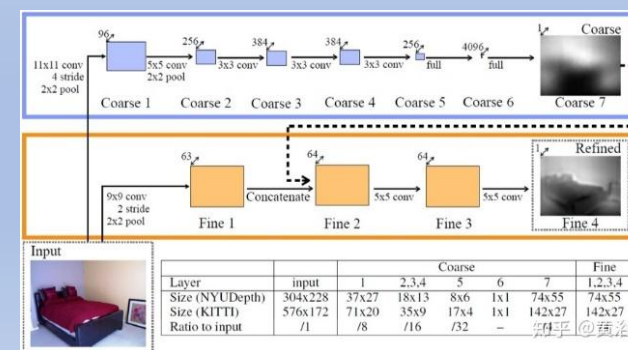
- shape from defocusing
- Shape from shading
- depth from geometrical perspective
- depth from haze
- shape from texture
- 基于学习:
- 2d-to-3d image conversion by learning depth from examples



基于成对训练集

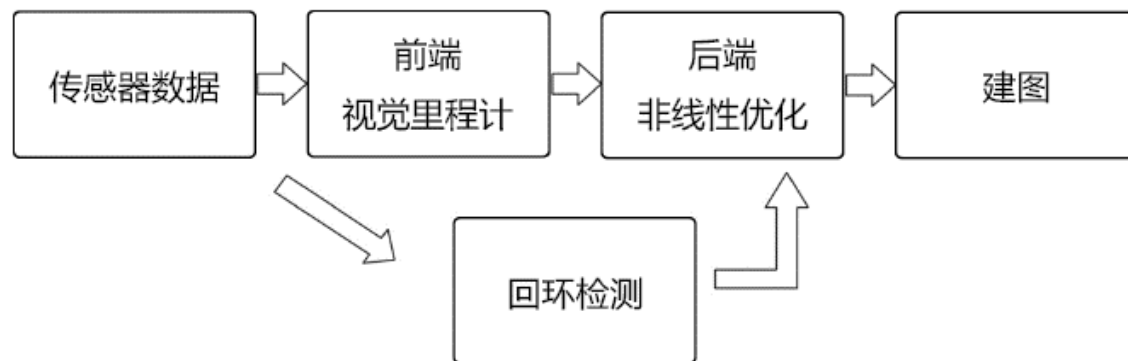


基于多帧运动信息



基于几何先验

视觉slam框架



视觉里程计 Visual Odometry

估算相机的帧间运动以及路标的位置

后端优化 Backend Optimization

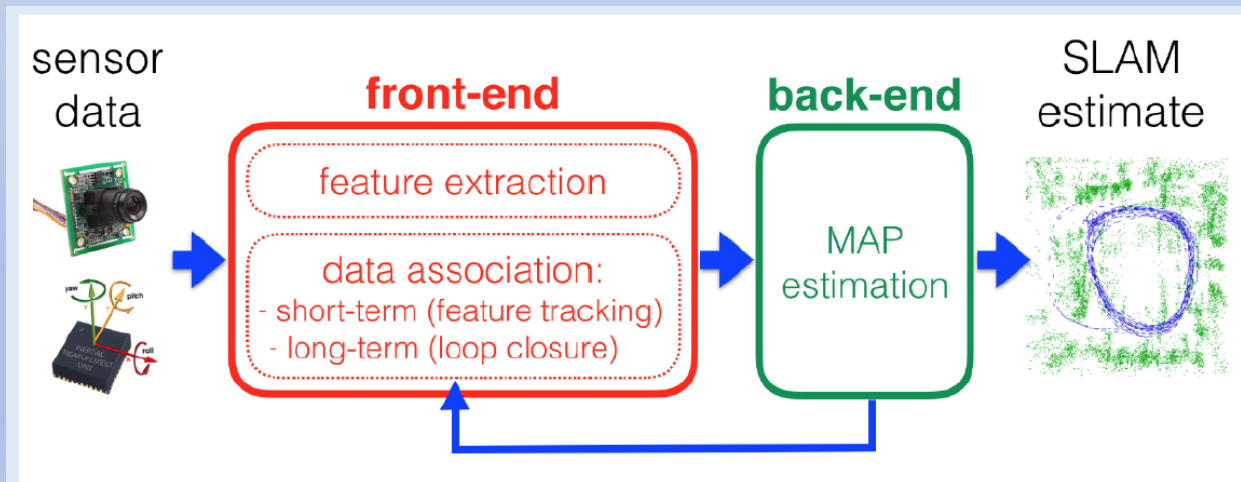
接受不同时刻视觉里程计测量的相机位姿，计算最大后验概率估计（MAP）

回环检测 Loop Closing

判断机器人是否曾经到达过先前的位置

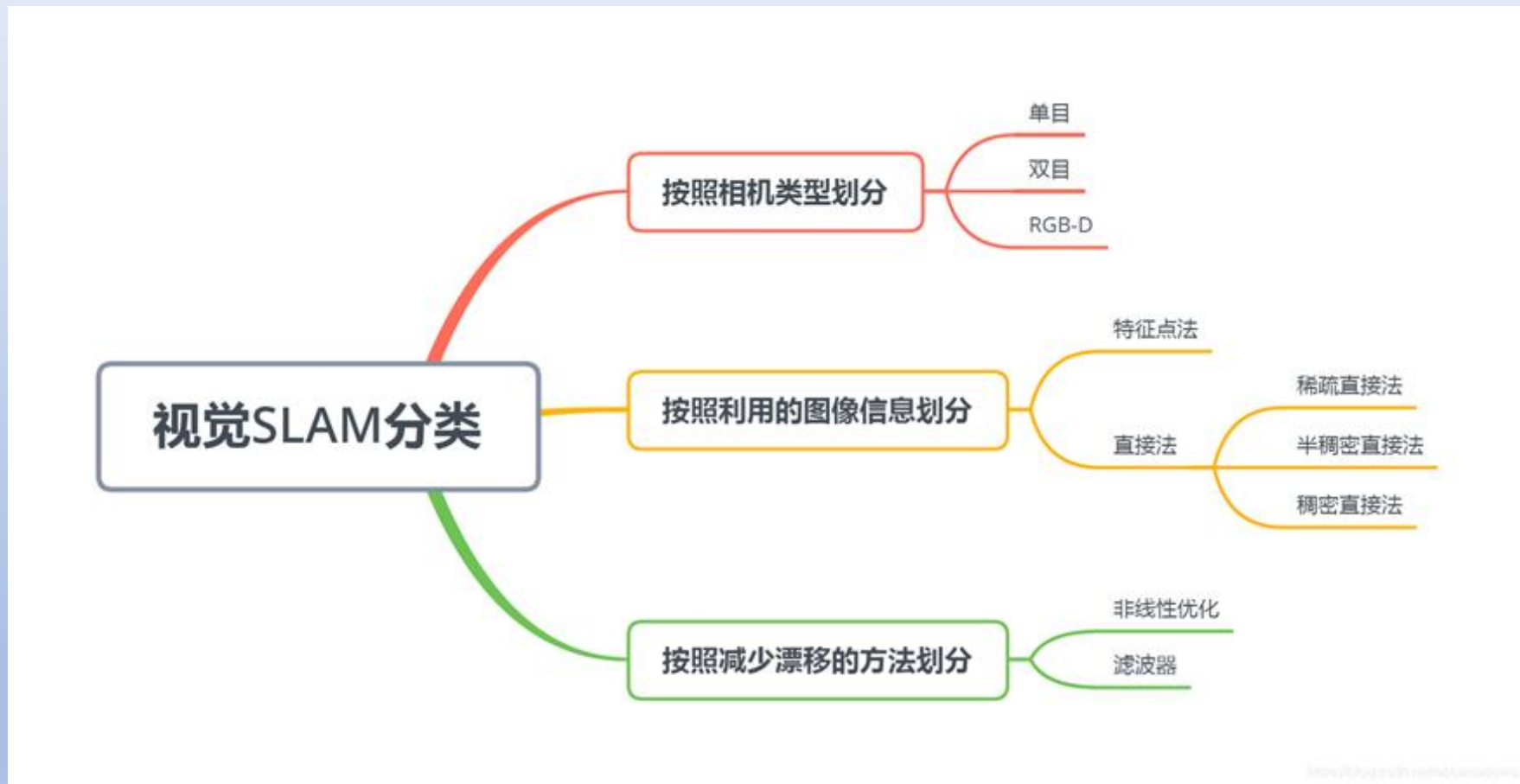
建图 Mapping

根据相机轨迹和图像，建立与任务要求对应的地图



SLAM is more like a concept than a single algorithm

视觉SLAM技术流派



相机与图像

• 小孔成像模型

原始形式

$$\frac{Z}{f} = -\frac{X}{X'} = -\frac{Y}{Y'}$$

翻转到前面

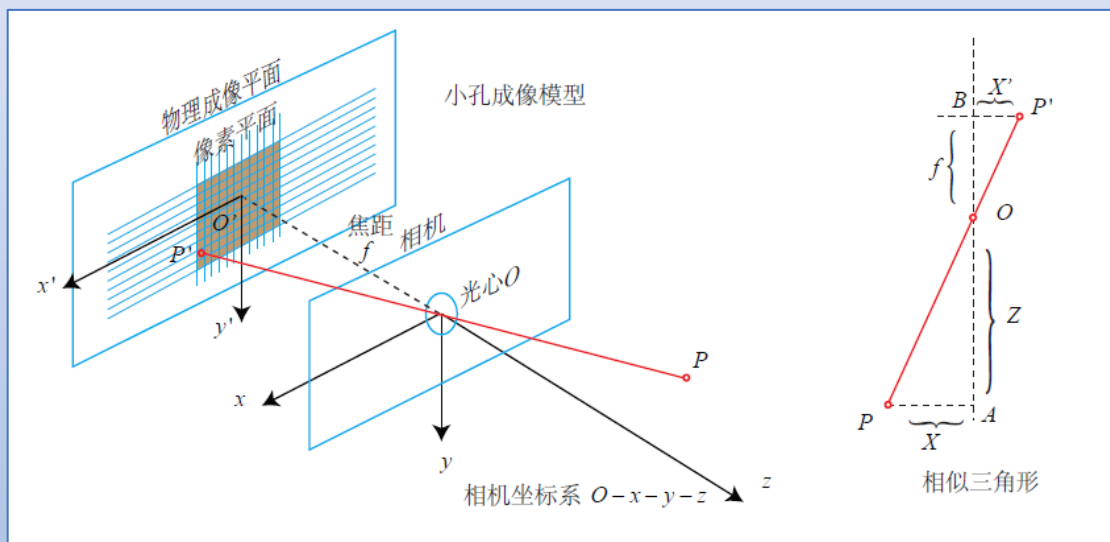
$$\frac{Z}{f} = \frac{X}{X'} = \frac{Y}{Y'}$$

整理之：

$$\begin{cases} X' = f \frac{X}{Z} \\ Y' = f \frac{Y}{Z} \end{cases}$$

最后考虑相机内参：

$$\begin{cases} u = f_x \frac{X}{Z} + c_x \\ v = f_y \frac{Y}{Z} + c_y \end{cases}$$



相机模型

$$\begin{cases} u = f_x \frac{X}{Z} + c_x \\ v = f_y \frac{Y}{Z} + c_y \end{cases}.$$

矩阵形式

$$\begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \frac{1}{Z} \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} X \\ Y \\ Z \end{pmatrix} \triangleq \frac{1}{Z} K P.$$

传统习惯

$$Z \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} X \\ Y \\ Z \end{pmatrix} \triangleq K P.$$

矩阵K称为内参数矩阵

给定相机和镜头，内参即固定

上述推导基于理想的小孔成像模型，真实镜头存在畸变，往往需要事先标定。

成像模型

- 除内参外，相机坐标系与世界坐标系还有一个变换T:

P点的世界坐标: P_w

P点的相机坐标: P_{uv}

$$Z P_{uv} = Z \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K (R P_w + t) = K T P_w.$$

- K为内参矩阵
- T为外参矩阵，这里 R, t 称为外参
- 外参是SLAM估计的目标

旋转矩阵R

同一个点，在具有旋转R、平移t的两个坐标系下的坐标转换。

1、旋转

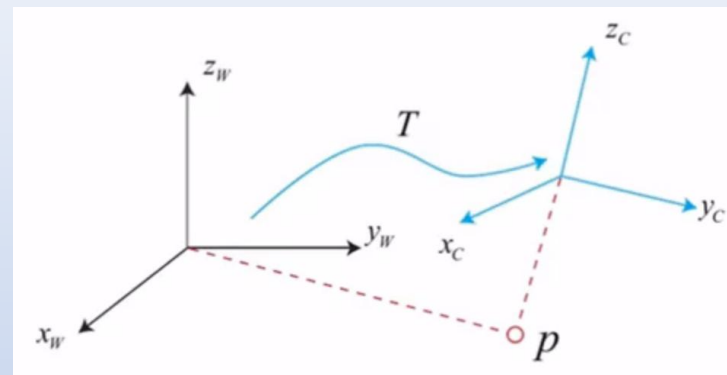
$$[e_1, e_2, e_3] \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = [e'_1, e'_2, e'_3] \begin{bmatrix} a'_1 \\ a'_2 \\ a'_3 \end{bmatrix}$$



$$\begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} e_1^T e'_1 & e_1^T e'_2 & e_1^T e'_3 \\ e_2^T e'_1 & e_2^T e'_2 & e_2^T e'_3 \\ e_3^T e'_1 & e_3^T e'_2 & e_3^T e'_3 \end{bmatrix} \begin{bmatrix} a'_1 \\ a'_2 \\ a'_3 \end{bmatrix} \triangleq R a'$$

2、加上平移

$$\begin{aligned} a_2 &= R_{21} \cdot a_1 + t_{21} \\ a_1 &= R_{12} \cdot a_2 + t_{12} \end{aligned}$$



则称 R 为旋转矩阵，其包含以下性质：

- R 是正交矩阵: $R^{-1} = R^T$, $R^T R = I$
- R 的行列式为1: $|R| = 1$

变换矩阵T: 齐次表示

- 若刚体在三维空间中发生多次旋转与平移

$$\mathbf{b} = \mathbf{R}_1 \mathbf{a} + \mathbf{t}_1, \quad \mathbf{c} = \mathbf{R}_2 \mathbf{b} + \mathbf{t}_2. \quad \implies \quad \mathbf{c} = \mathbf{R}_2 (\mathbf{R}_1 \mathbf{a} + \mathbf{t}_1) + \mathbf{t}_2.$$

为了方便表达这种连续运动, 引入了齐次坐标形式表示旋转和平移

$$\begin{bmatrix} \mathbf{a}' \\ 1 \end{bmatrix} = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0}^T & 1 \end{bmatrix} \begin{bmatrix} \mathbf{a} \\ 1 \end{bmatrix} \triangleq \mathbf{T} \begin{bmatrix} \mathbf{a} \\ 1 \end{bmatrix}$$

T 为变换矩阵
(同时包含旋转R与平移t)

则多次的旋转与平移可以如式连乘:

$$\tilde{\mathbf{b}} = \mathbf{T}_1 \tilde{\mathbf{a}}, \tilde{\mathbf{c}} = \mathbf{T}_2 \tilde{\mathbf{b}} \quad \Rightarrow \quad \tilde{\mathbf{c}} = \mathbf{T}_2 \mathbf{T}_1 \tilde{\mathbf{a}}$$

欧拉角(Euler Angles)

- 旋转矩阵R有9个元素，但是仅仅表示3个自由度的信息，不节省内存空间。
- 并且旋转矩阵还需要满足一系列约束条件。

$$\begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} \mathbf{e}_1^T \mathbf{e}'_1 & \mathbf{e}_1^T \mathbf{e}'_2 & \mathbf{e}_1^T \mathbf{e}'_3 \\ \mathbf{e}_2^T \mathbf{e}'_1 & \mathbf{e}_2^T \mathbf{e}'_2 & \mathbf{e}_2^T \mathbf{e}'_3 \\ \mathbf{e}_3^T \mathbf{e}'_1 & \mathbf{e}_3^T \mathbf{e}'_2 & \mathbf{e}_3^T \mathbf{e}'_3 \end{bmatrix} \begin{bmatrix} a'_1 \\ a'_2 \\ a'_3 \end{bmatrix} \triangleq \mathbf{R} \mathbf{a}'$$

- R是正交矩阵: $R^{-1} = R^T$, $R^T R = I$
- R的行列式为1: $|R| = 1$

- 所以，利用欧拉角 (Φ , θ , ψ) 表示三个方向上的旋转，最常用绕动轴的Z-Y-X旋转方式:

$$R_{b/mv} = R_\phi R_\theta R_\psi = \begin{bmatrix} \cos \theta \cos \psi & \cos \theta \sin \psi & -\sin \theta \\ \sin \phi \sin \theta \cos \psi - \cos \phi \sin \psi & \sin \phi \sin \theta \sin \psi + \cos \phi \cos \psi & \sin \phi \cos \theta \\ \cos \phi \sin \theta \cos \psi + \sin \phi \sin \psi & \cos \phi \sin \theta \sin \psi - \sin \phi \cos \psi & \cos \phi \cos \theta \end{bmatrix}$$

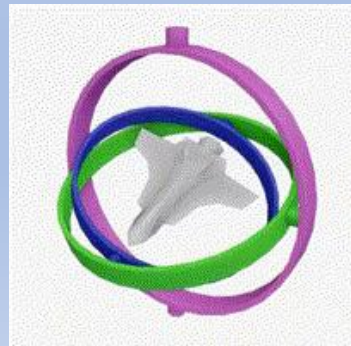
优点：直观、紧凑。

缺点：存在奇异性，不适合做插值或迭代。

万向锁问题：仰角达到 $\pm 90^\circ$ 时，在空间中失去了一个自由度。

当 $\theta = 90^\circ$ 时， $\cos(\theta) = 0$, $\sin(\theta) = 1$ 。

$$R = \begin{bmatrix} 0 & 0 & 1 \\ \sin(\phi + \psi) & \cos(\phi + \psi) & 0 \\ -\cos(\phi + \psi) & \sin(\phi + \psi) & 0 \end{bmatrix}$$

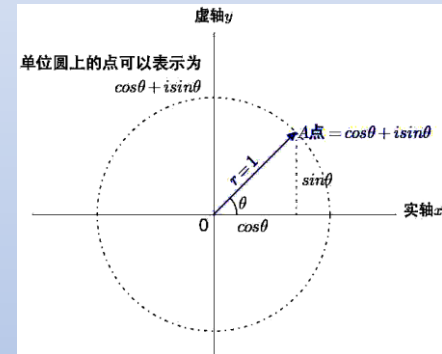


四元数

- 为了解决旋转矩阵浪费空间、欧拉角奇异性等问题，提出四元数方法，是一种既节省空间，又不存在奇异性的刚体旋转表示方式。

我们在复数平面坐标系下，可以用单位复数来表达旋转操作

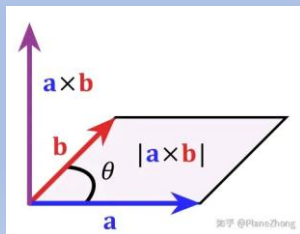
$$z = x + iy = \rho e^{i\theta}$$



四元数:复数扩充到三维:

$$\mathbf{q} = q_0 + q_1 i + q_2 j + q_3 k, \quad \mathbf{q} = [s, \mathbf{v}], \quad s = q_0 \in \mathbb{R}, \mathbf{v} = [q_1, q_2, q_3]^T \in \mathbb{R}^3$$

$$\begin{cases} i^2 = j^2 = k^2 = -1 \\ ij = k, ji = -k \\ jk = i, kj = -i \\ ki = j, ik = -j \end{cases}$$



使用四元数表示三维空间中的旋转

若点 p 经过一次以四元数 q 表示的旋转后得 p' ，包含以下两个步骤：

1. 将 p 坐标用纯虚的四元数表示: $p = [0, x, y, z]$
2. 使用四元数 q 将其旋转(前面乘以 q ，后面乘以 q^{-1}):

$$p' = qpq^{-1}$$

完备，但不直观

虚部之间：自己与自己运算像复数运算，自己与别人运算像叉乘（法向量方向）

李群

- 旋转矩阵 R 、变换矩阵 T 最为常用，但是 $R1+R2$ 的结果，不是旋转矩阵， T 亦然。
- 即： R 、 T 对乘法封闭，但是对加法不封闭，因此无法计算微分。

$$\min_{\mathbf{T}} J(\mathbf{T}) = \sum_{i=1}^N \|\mathbf{z}_i - \mathbf{T}\mathbf{p}_i\|_2^2$$

例如：目标函数 $J(T)$ ，要求最佳的 T ，就需要对 T 求导。
加法不封闭，则无法求导：

$$f'(x) = \lim_{\Delta x \rightarrow 0} \frac{f(x+\Delta x) - f(x)}{\Delta x}$$

旋转/变换矩阵代数结构上不够好，于是成群。

群 (Group) 是一种集合加上一种运算的代数结构。

李群(lie group): 指具有连续（光滑）性质的群，是群也是流形。刚体可以在空间中连续运动，故三维旋转矩阵 $SO(3)$ 和三维变换矩阵 $SE(3)$ 都是李群。

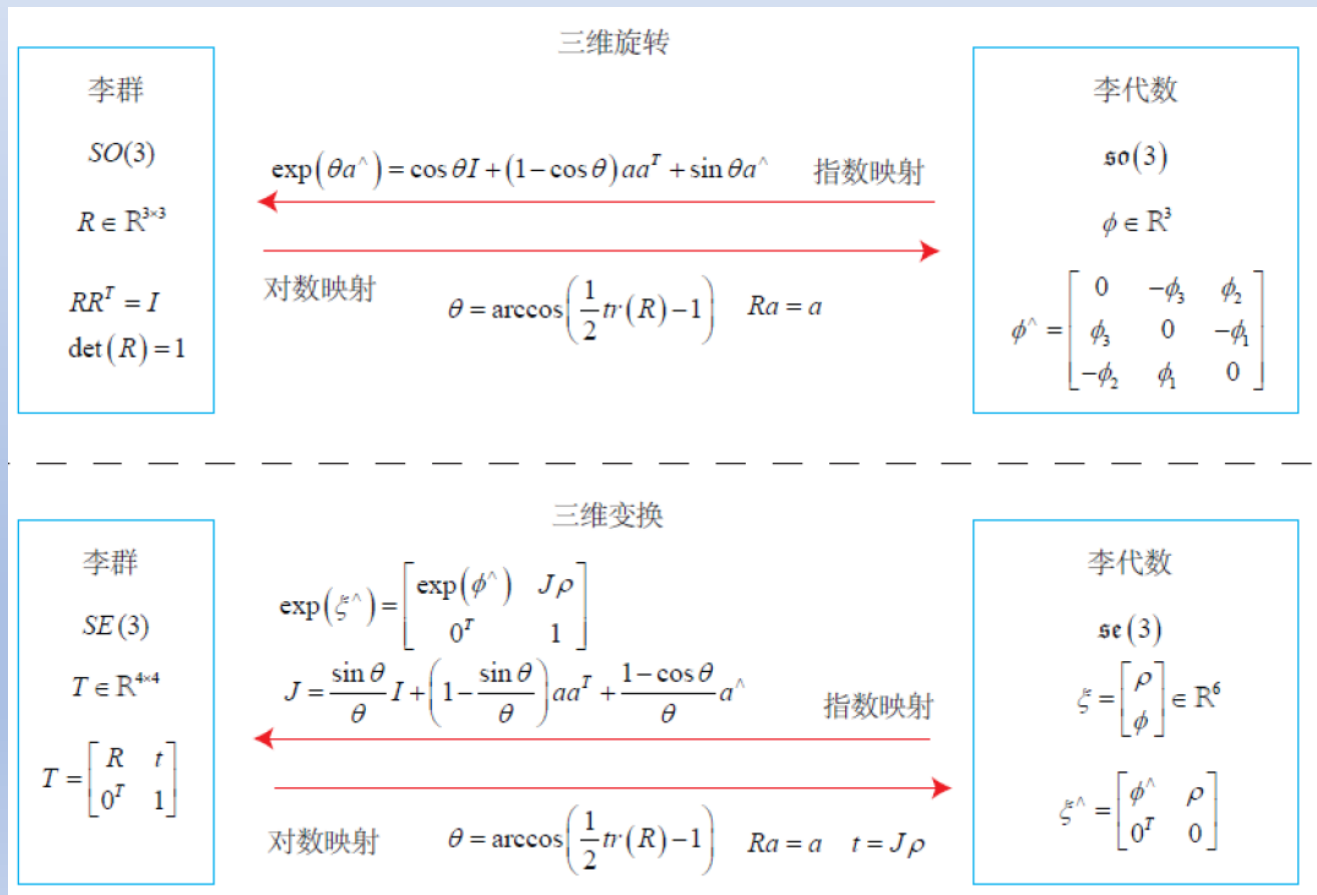
$$SO(3) = \{\mathbf{R} \in \mathbb{R}^{n \times n} | \mathbf{R}\mathbf{R}^T = \mathbf{I}, \det(\mathbf{R}) = 1\}$$

$$SE(3) = \left\{ \mathbf{T} = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0}^T & 1 \end{bmatrix} \in \mathbb{R}^{4 \times 4} | \mathbf{R} \in SO(3), \mathbf{t} \in \mathbb{R}^3 \right\}$$

需求：SLAM中需要对相机位姿迭代微调

李群和李代数

李代数：李群单位元处的正切空间，每个李群都有与之对应的李代数。用小写 $\mathfrak{so}(3)$ 和 $\mathfrak{se}(3)$ 表示。通过BCH线性近似，可以定义加法，从而在李代数上的求导。

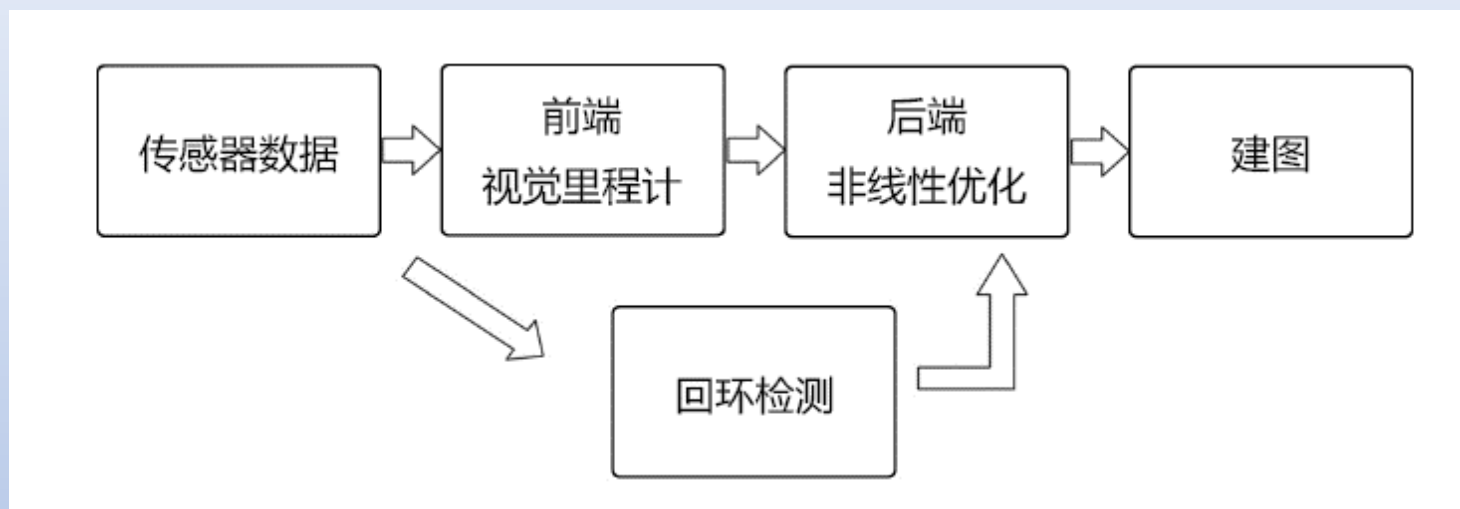


导数存在两种解法

- 1) 对 R 对应的李代数加上小量，求相对于小量的变化率（导数模型）；
- 2) 对 R 左乘或右乘一个小量，求相对于小量的李代数的变化率（扰动模型）

程序库：Eigen, Sophus, tf

视觉slam框架



视觉里程计Visual Odometry

估算相机的帧间运动以及路标的位置

后端优化 Backend Optimization

接受不同时刻视觉里程计测量的相机位姿，计算最大后验概率估计（MAP）

回环检测 Loop Closing

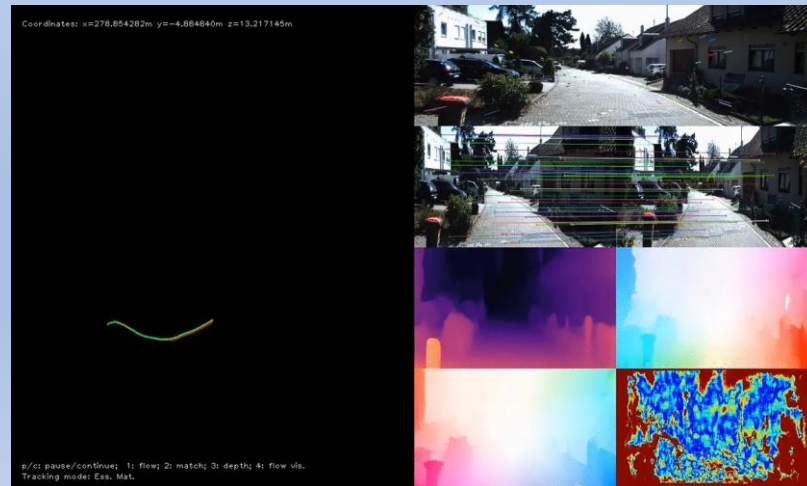
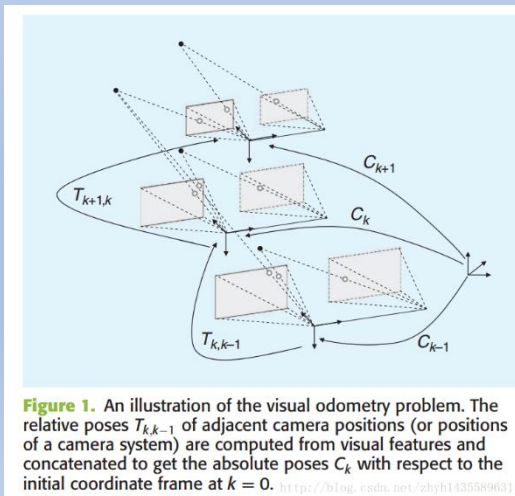
判断机器人是否曾经到达过先前的位置

建图Mapping

根据相机轨迹和图像，建立与任务要求对应的地图

视觉里程计 (VO)

- **VO目的：通过两张图像的成像几何约束，估计相机运动。**
- **方法：特征点法，直接法**
 - **特征点方法目前占据主流**，能够在噪声较大、相机运动较快时工作，但地图是稀疏特征点；
 - 直接方法不需要提特征，能够建立稠密地图，但存在着计算量大、鲁棒性不好的缺陷。
 - 基于深度学习的vo工作越来越多：有的直接用cnn拟合pose（deepvo），有的使用多帧之间的视差估计运动（geonet），也有基于光流的（vinet，DF-VO）。



VO本质是图像配准技术



特征空间	相似性度量	搜索策略
灰度	相关系数	层次
特征点	归一化相关系数	迭代点匹配
边缘强度	相关系数与匹配滤波器	层次迭代或模拟退火
二值边缘	统计相关与匹配滤波器	模拟退火
曲线	绝对差和, 局部熵差	遗传算法
表面	掩模相关	树或图匹配
统计属性	Hausdorff距离	松弛算法
模型	最小距离分类器	能量最少化
...		

图像配准三要素

特征点法

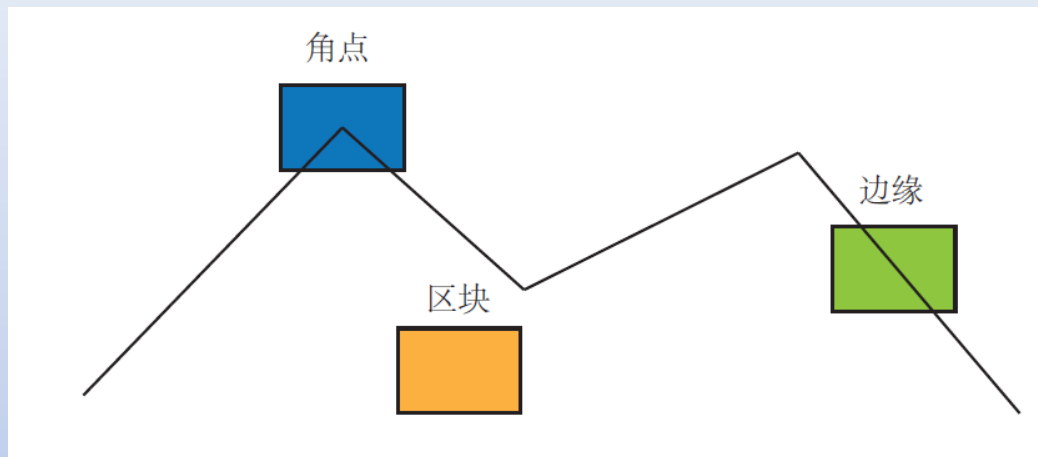
- 经典SLAM模型中以**位姿-路标** (Landmark) 来描述SLAM过程
- **路标**是三维空间中固定不变的点，能够在特定位姿下观测到
 - 数量充足，以实现良好的定位
 - 较好的区分性，以实现数据关联
- 在视觉SLAM中，一般利用**图像特征点**作为SLAM中的路标



Figure 1. We present a new method to recognize scene-specific *scene landmarks* to localize a camera, which preserves privacy and achieves high accuracy. [Left] Scene landmark detections in a query image obtained from a heatmap-based CNN architecture. [Middle] A visualization of the predicted heatmap scores. [Right] The 3D scene landmarks (in red) and the estimated camera pose (in blue) are shown over the 3D point cloud (in gray). The 3D point cloud is shown only for the purpose of visualization.

特征点提取

- **特征点提取**：图像当中具有代表性的关键部分
 - 可重复性
 - 可区别性
 - 高效
 - 局部
- **特征点的描述**
 - 位置、大小、方向、评分等
 - 特征点领域的信息—描述子 (Descriptor)



点-线-面

定位精度	递减
定姿能力	递增
唯一性	递增

特征描述希望在光照、视角发生少量变化时仍能保持一致

用于配准的图像描述子

- 人工设计检测算法：sift、surf、orb、fast、hog
- 基于深度学习的方法：人脸关键点检测、3D match点云匹配
- 场景中的人工标记点：简单的标记，二维码设计。。



(OpenCV features2d模块)

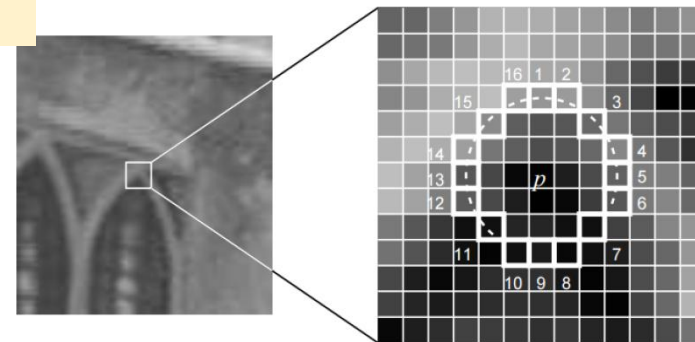
ORB特征 (Oriented FAST and Rotated BRIEF)

- ORB是目前最快速稳定的特征点检测和提取算法。

ORB要比SIFT快两个数量级!

- 寻找特征点FAST: p 为圆心、半径为3的圆上, 连续 N 个点的灰度大于 p 一定程度(阈值), 即为特征点
- 旋转不变性Oriented FAST: 在FAST基础上计算几何中心和质心的向量作为主方向
- 关键点描述BRIEF: 按照一定模式(例如高斯分布), 在特征点邻域取128对点(BRIEF128)比较大小。得到128维二进制字符串010...01010

1



- 在一个小的图像块 B 中, 定义图像块的矩为:

$$m_{pq} = \sum_{x,y \in B} x^p y^q I(x,y), \quad p, q = \{0,1\}.$$

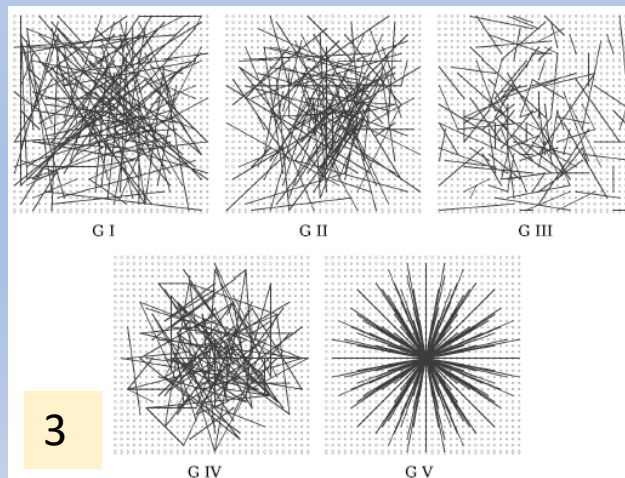
- 通过矩可以找到图像块的质心:

$$C = \left(\frac{m_{10}}{m_{00}}, \frac{m_{01}}{m_{00}} \right).$$

- 连接图像块的几何中心 O 与质心 C , 得到一个方向向量 $\overrightarrow{OC}$, 于是特征点的方向可以定义为:

$$\theta = \arctan(m_{01}/m_{10}).$$

2



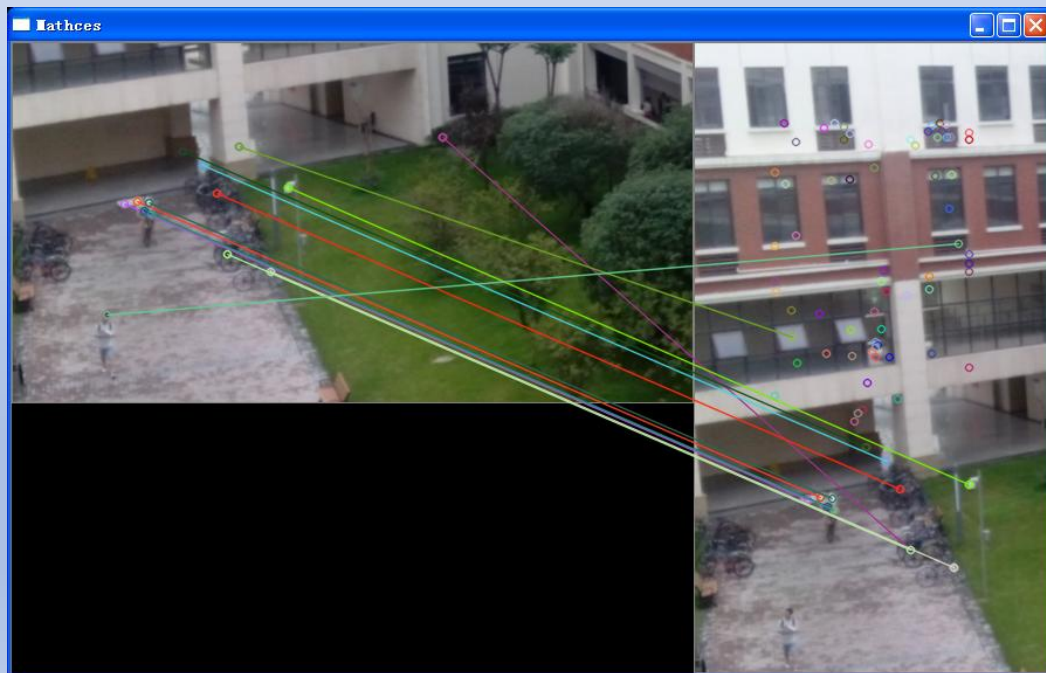
3

128-512 bit string

Binary Feature Vectors $V_1 = [01011100100110...]$

ORB特征匹配

- 通过描述子的差异判断两幅图像中的同名点。
- 方法：比较图1中每个特征和图2每个特征的距离。



汉明距离(Hamming Distance)适合计算长度相等两个向量的相似度：

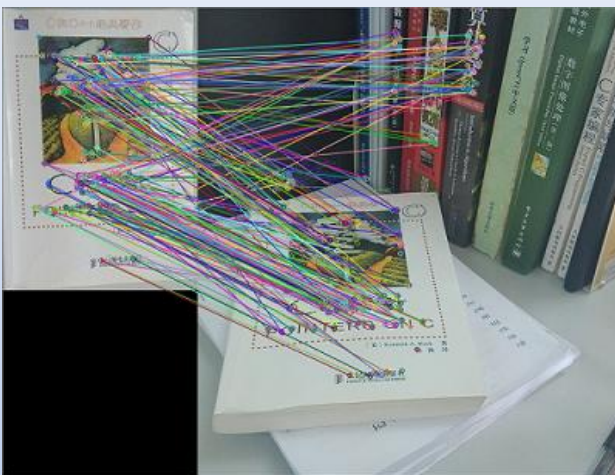
比较向量的每一位是否相同，若不同则汉明距离加1，向量相似度越高，对应的汉明距离越小。

a=11101010

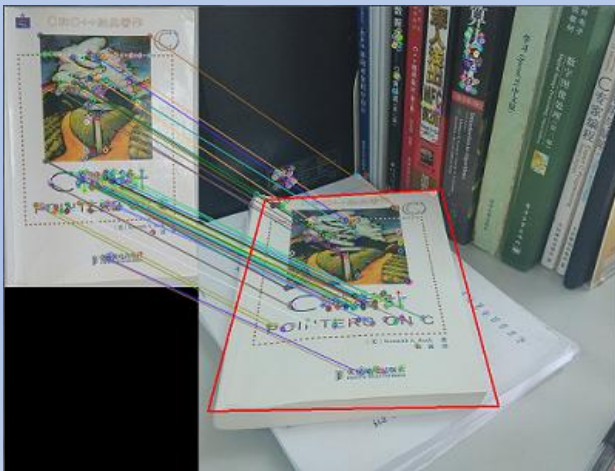
b=11011010

汉明距离多少？ **2**

RANSAC—致性筛选

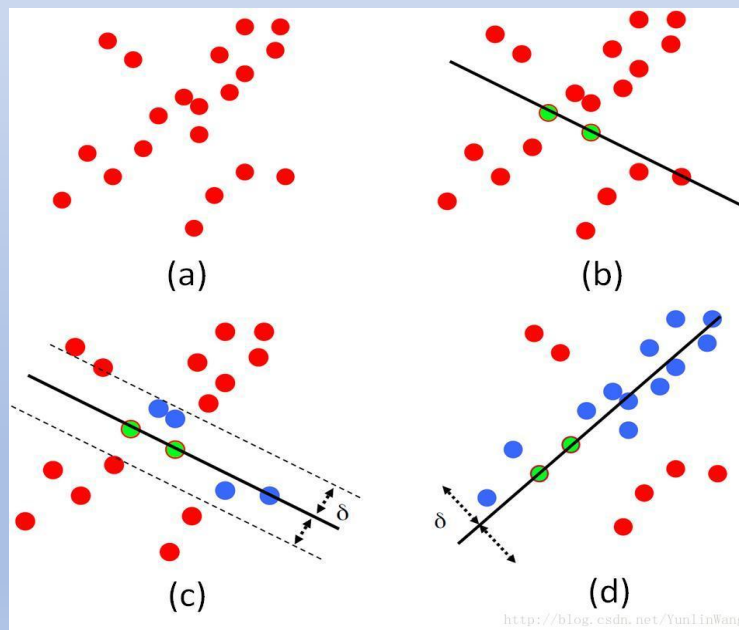


RANSAC



原始的特征点匹配一般会有大量误匹配关系

RANSAC(Random Sample Consensus,随机采样一致)通过使用观测数据的随机样本来估计模型参数，使用投票机制来寻找优化的拟合结果。



假设数据是由“内点”和“外点”组成的。

“内点”就是组成模型参数的数据，“外点”就是不适合模型的数据。

如果估计的模型中，内点数量/比例高于某个阈值，就可以停止迭代。

随机选2点（绿点）构成直线模型，
则蓝色是内点，红色为外点。

RANSAC分析

- 算法优势

- 离群点鲁棒性：即使数据中包含大量离群点，也能准确拟合模型
- 模型通用性：适用于多种几何模型拟合（直线、平面、球面...）
- 理论保证：有数学理论支持，可以计算所需迭代次数
- 参数直观：参数有明确的物理意义

- 算法局限性

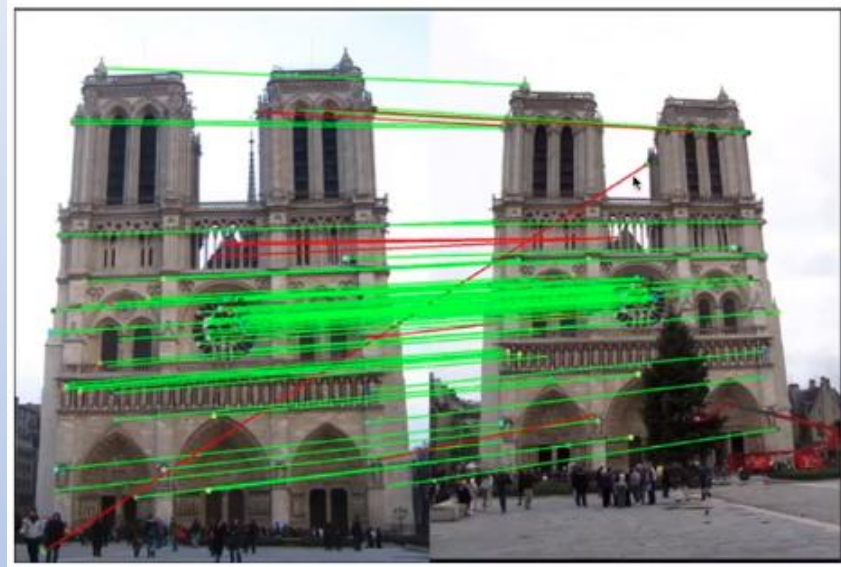
- 计算效率：对于低内点比例的数据，需要大量迭代
- 模型假设：需要预先知道要拟合的模型类型
- 参数敏感性：距离阈值的选择对结果影响较大
- 局部最优：可能陷入局部最优解，特别是当数据中存在多个模型时

内点距离门限

- 过小：可能遗漏真正内点，导致模型拟合不准确
- 过大：可能包含太多外点，降低模型精度
- 假设特征点定位误差维为 σ
- 对于 1D 残差（点到直线距离）
置信度（0.95） 阈值 $\approx 1.96\sigma$
置信度（0.99） 阈值 $\approx 2.58\sigma$

特征点对应->空间变换

- 经过特征匹配和一致性筛选之后，得到两幅图像特征点之间的对应关系，下一步需要估计出相机的空间运动（位姿）。



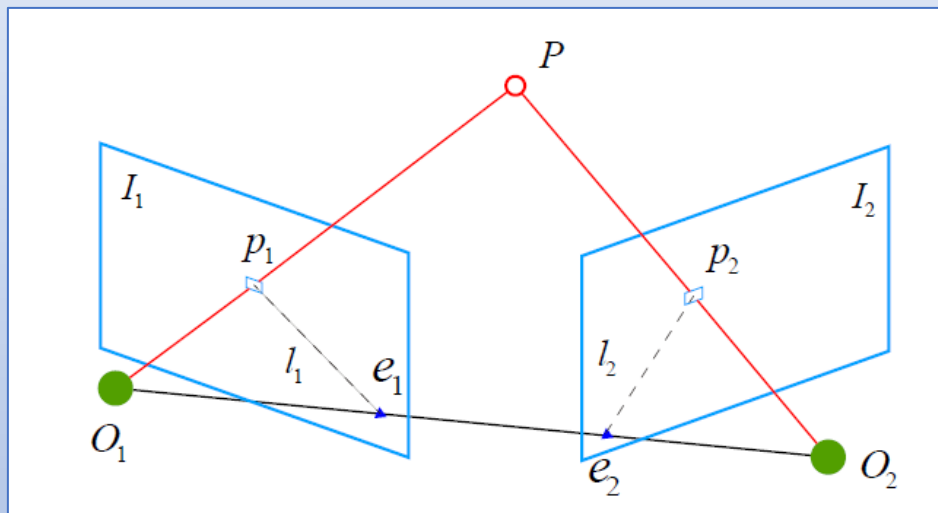
- 如果只有两个单目图像，得到2D-2D间的关系—对极几何：只根据两组像素位置，估计运动
- 如果匹配的是帧和地图，得到3D-2D间的关系—PnP：已知空间位置和投影位置，估计运动
- 如果匹配的是RGB-D图，得到3D-3D间的关系—ICP：已知两组空间位置，估计运动

对极几何约束 (图像-图像)

对极几何描述的是两个相机在不同位置拍摄同一场景时，图像点之间的几何关系。

几何关系:

- P 在两个图像的投影为 p_1, p_2
- 两个相机之间的变换为 T_{12}
- O_1P 在第二个图像上投影为 e_2p_2
 - 记 l_2 ，称为极线，反之亦然
- e_1, e_2 称为极点



$p_1, p_2, O_1, O_2, l_1, l_2$ 共面

有对极几何约束

Essential本质矩阵表征了同一物点在两个透视相机**成像平面**上像点的几何约束关系。

Fundamental基本矩阵表征了同一物点在两个透视相机**像素面**上像素点间的几何约束关系

$$x_2^T \underline{t}^\wedge R x_1 = 0.$$

成像平面

$$p_2^T K^{-T} \underline{t}^\wedge R K^{-1} p_1 = 0.$$

像素平面

$$E = \underline{t}^\wedge R, \quad F = K^{-T} E K^{-1},$$

$$\begin{bmatrix} 0 & -t_z & t_y \\ t_z & 0 & -t_x \\ -t_y & t_x & 0 \end{bmatrix}$$

$\underline{t}$ 的反对称矩阵 K 为相机内参

对极几何求解

1. 求解本质矩阵E

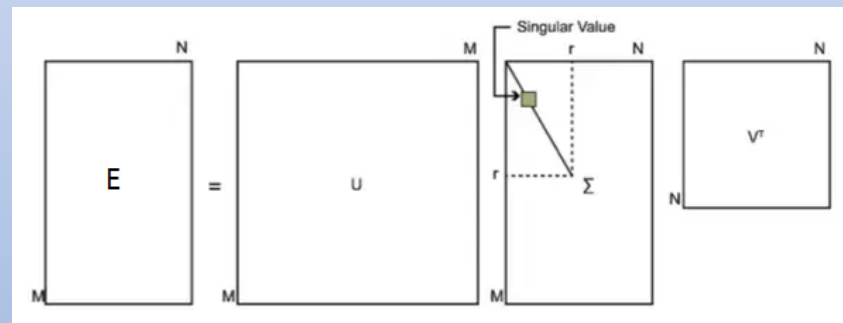
$$E = t^{\wedge} R,$$

E矩阵有5个自由度 (旋转3+平移3-尺度1)
理论上5个点即可求解, 一般常用八点法,
用最小二乘或RANSAC求解E。

$$\begin{pmatrix} u_1 & v_1 & 1 \end{pmatrix} \begin{pmatrix} e_1 & e_2 & e_3 \\ e_4 & e_5 & e_6 \\ e_7 & e_8 & e_9 \end{pmatrix} \begin{pmatrix} u_2 \\ v_2 \\ 1 \end{pmatrix} = 0.$$

2. 对E矩阵进行SVD奇异值分解

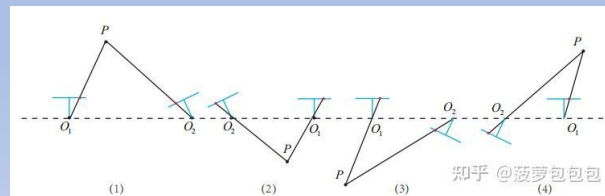
$$E = U \Sigma V^T,$$



3. 得到R,t:

$$\begin{aligned} t_1^{\wedge} &= U R_Z(\frac{\pi}{2}) \Sigma U^T, & R_1 &= U R_Z^T(\frac{\pi}{2}) V^T \\ t_2^{\wedge} &= U R_Z(-\frac{\pi}{2}) \Sigma U^T, & R_2 &= U R_Z^T(-\frac{\pi}{2}) V^T. \end{aligned}$$

消除歧义



参考: <https://zhuanlan.zhihu.com/p/368843742>

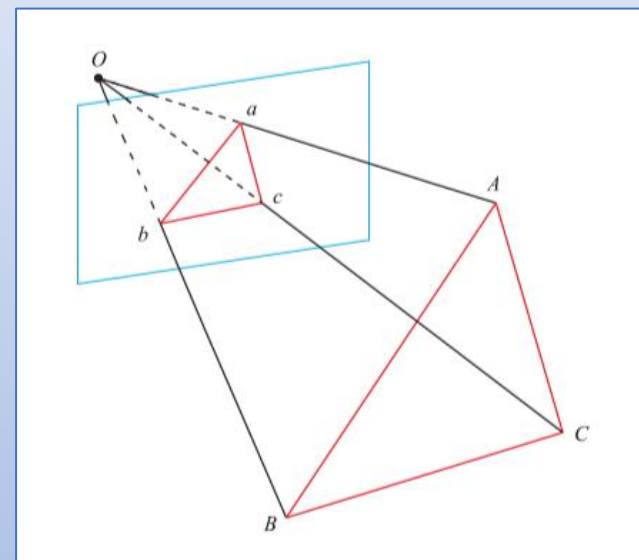
pnp约束 (图像-地图)

- PnP (**Perspective-n-Point**) 求解2D相机坐标系p相对3D世界坐标系P的位姿 (R, t)。

小孔
成像
模型

$$\omega \mathbf{p} = Z^C \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \begin{pmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} X^C \\ Y^C \\ Z^C \end{pmatrix}$$

$$= \begin{pmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} r_{11} & r_{12} & r_{13} & t_1 \\ r_{21} & r_{22} & r_{23} & t_2 \\ r_{31} & r_{32} & r_{33} & t_3 \end{pmatrix} \begin{pmatrix} X^W \\ Y^W \\ Z^W \\ 1 \end{pmatrix} = K[R|t]P^W$$



PnP就是在已知 n 个点的像素坐标 p_i 及其世界坐标系下的坐标 P_i^W 和相机的内参矩阵 K 的前提下, 求解相机的位姿 R 和 t 。

Pnp求解 - DLT

- DLT（直接线性变换）
 - 设空间点 $P = (X, Y, Z, 1)^T$
 - 投影点为: $x = (u, v, 1)$
 - 投影关系: $\omega x = [R|t]p$

$$\omega \begin{pmatrix} u_1 \\ v_1 \\ 1 \end{pmatrix} = \begin{pmatrix} t_1 & t_2 & t_3 & t_4 \\ t_5 & t_6 & t_7 & t_8 \\ t_9 & t_{10} & t_{11} & t_{12} \end{pmatrix} \begin{pmatrix} X^W \\ Y^W \\ Z^W \\ 1 \end{pmatrix}$$

将它看成一个矩阵T的线性方程，求解

利用最后一行把 ω 消去，可得两个约束：

$$u_1 = \frac{t_1 X_W + t_2 Y_W + t_3 Z_W + t_4}{t_9 X_W + t_{10} Y_W + t_{11} Z_W + t_{12}}$$

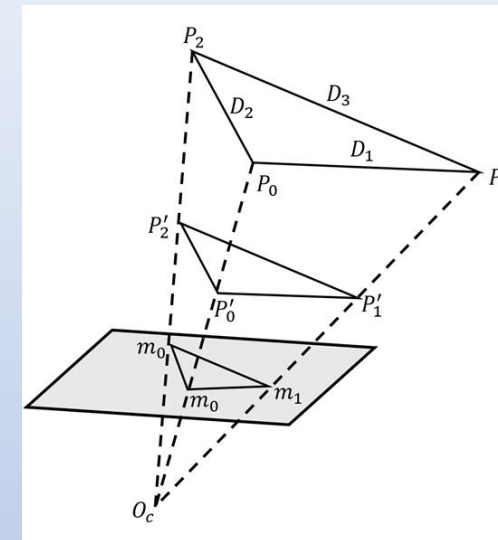
$$v_1 = \frac{t_5 X_W + t_6 Y_W + t_7 Z_W + t_8}{t_9 X_W + t_{10} Y_W + t_{11} Z_W + t_{12}}$$

DLT方法：每个特征点提供了两个关于T的线性约束，T一共有12维，最少通过6对匹配点即可实现增广矩阵 $[R|t]$ 的线性求解。

如果匹配点个数大于6，通过最小二乘求最优。

其他pnp求解方法

- **P3P方法**：将世界坐标系下3个点的3D坐标转换到相机坐标系下的3D坐标，之后将问题转化为**3D-3D的ICP问题**。



- **最小化重投影误差方法**：世界坐标系下的3D点投影到2D图像，根据对应点的重投影误差构建非线性优化问题。

$$\mathbf{T}^* = \underset{\mathbf{T}}{\operatorname{argmin}} \frac{1}{2} \sum_{i=1}^n \left\| \mathbf{p}_i - \frac{1}{\omega_i} \mathbf{K} \mathbf{T} \mathbf{P}_i^W \right\|^2$$

利用李代数得到误差关于位姿的导数以指导优化方向，迭代求解

其他视觉里程计方法

- 基于特征的VO方法的耗时估算：

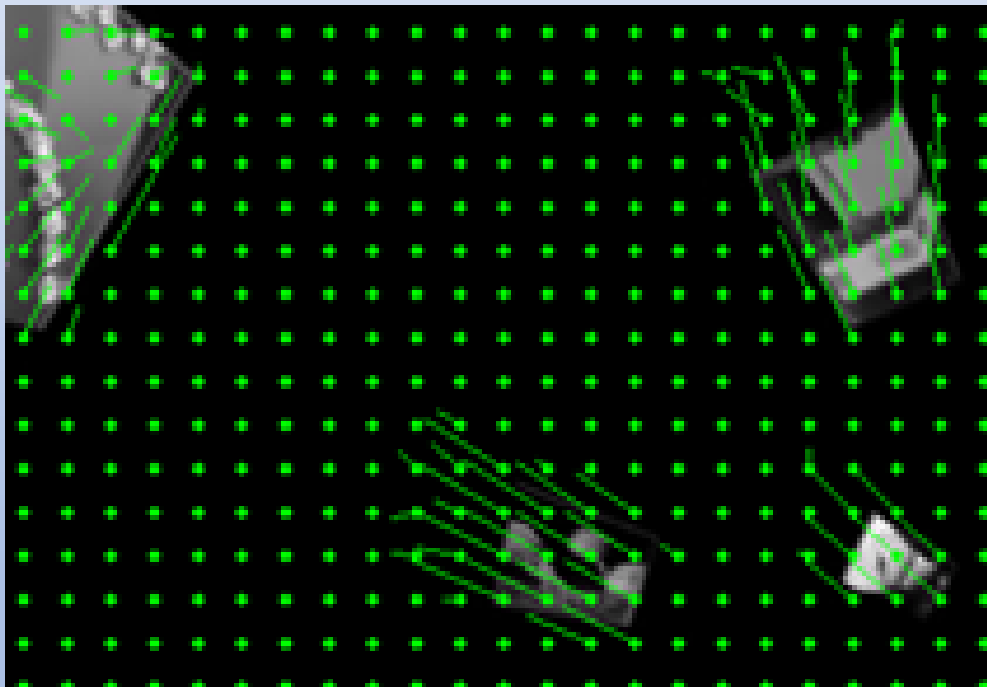
- | | |
|---------------------|-------------------|
| 1. 在图像中提取特征点并计算特征描述 | 非常耗时 10+ms in ORB |
| 2. 在不同图像中寻找特征匹配 | 非常耗时 $O(n^2)$ |
| 3. 利用匹配点信息计算相机位姿 | 快速 < 1ms |

- 不计算特征计算VO的思路：

1. 通过其他方式寻找配对点
光流
2. 无配对点
直接法

光流 (optical flow)

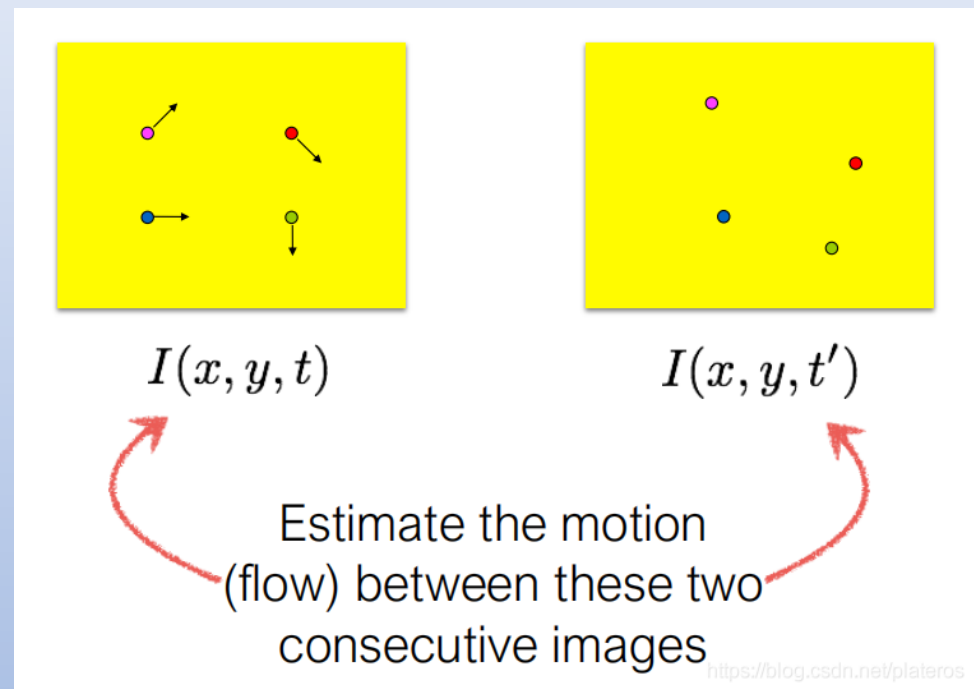
光流是空间运动物体在成像平面上的像素运动的瞬时速度。



光流是由于场景中物体本身的移动、相机的运动，或者两者的共同运动所产生的

光流方法

- 光流法：利用图像序列中像素在时间域上的变化以及相邻帧之间的相关性，来找到上一帧跟当前帧之间存在的对应关系，从而计算出相邻帧之间相机、物体的运动信息的一种方法。



所谓光流就是每个像素点瞬时速率，在时间间隔很小（比如视频的连续前后两帧之间）时，也等同于目标点的位移

两种光流

- 一般分为稀疏光流（关键点）和稠密光流（逐像素）
 - 稀疏以Lucas-Kanade（LK）光流为代表
 - 稠密以Horn-Schunck（HS）光流为代表



稀疏光流



稠密光流

光流计算

前提条件:

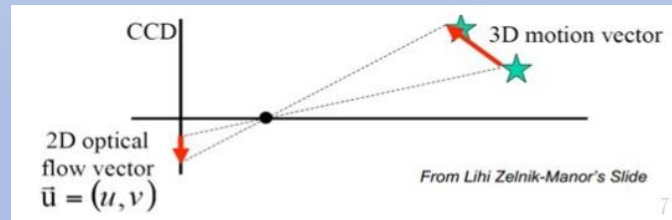
- (1) **亮度恒定不变**。即同一目标在不同帧间运动时，其亮度不会发生改变。这是基本光流法的假定（所有光流法变种都必须满足），用于得到光流法基本方程；
- (2) **时间连续或运动是“小运动”**。即目标随时间连续、光滑运动，且相邻帧之间位移要比较小。同样也是光流可以估计的前提。

- 设 t 时刻位于 x, y 处像素点的灰度值为
- 在 $t+dt$ 时刻，该像素运动到了
希望计算运动 dx, dy
- 灰度不变假设

$$I(x, y, t).$$

$$I(x + dx, y + dy, t + dt)$$

$$I(x + dx, y + dy, t + dt) = I(x, y, t).$$



灰度不变是一种理想的假设，由于高光/阴影/材质/曝光等不同，很可能不成立

光流推导

- 对 $t+dt$ 时刻的灰度进行Taylor展开并保留一阶项:

$$I(x + dx, y + dy, t + dt) \approx I(x, y, t) + \frac{\partial I}{\partial x}dx + \frac{\partial I}{\partial y}dy + \frac{\partial I}{\partial t}dt.$$

- 由于灰度不变, 所以

$$\frac{\partial I}{\partial x}dx + \frac{\partial I}{\partial y}dy + \frac{\partial I}{\partial t}dt = 0.$$

因此 =>

$$\frac{\partial I}{\partial x} \frac{dx}{dt} + \frac{\partial I}{\partial y} \frac{dy}{dt} = -\frac{\partial I}{\partial t}.$$

帧内
x
方向
梯度

速度
U

帧内
y
方向
梯度

速度
V

帧间
灰度
变化
率

求解: $u=dx/dt$ $v=dy/dt$

光流计算

$$\frac{\partial I}{\partial x} \frac{dx}{dt} + \frac{\partial I}{\partial y} \frac{dy}{dt} = -\frac{\partial I}{\partial t}.$$

约束方程只有一个，而方程的未知量有两个 dx/dt , dy/dt （欠定）。此时需要引入另外的约束条件，从不同的角度引入约束条件，导致了不同光流场计算方法。

目前有四类方法：基于梯度（微分）的方法、基于匹配的方法、基于能量（频率）的方法、基于相位的方法和神经动力学方法。

梯度方法由于计算简单和较好的结果，得到了广泛应用和研究。典型的代表是Lucas-Kanade(LK) 算法与Horn-Schunck算法。

Lucas-Kanade(LK) 算法

算法适用于稀疏特征点处计算运动向量。

- **空间一致性假设：** 场景中相同表面的相邻点具有相似的运动，并且其投影到图像平面上的距离也比较近。

假设在一个大小为 $m \times m(n)$ 的窗口内，图像的光流是一个恒定值。那么就可以得到方程组：

$$I_x u + I_y v + I_t = 0$$

$$\begin{bmatrix} I_{x1} & I_{y1} \\ I_{x2} & I_{y2} \\ \vdots & \vdots \\ I_{xn} & I_{yn} \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} -I_{t1} \\ -I_{t2} \\ \vdots \\ -I_{tn} \end{bmatrix}$$

记做 $A\vec{V} = -b$, 采用最小二乘法得到

$$A^T A \vec{V} = A^T (-b)$$

则最终所有求解的光流（速度矢量）为：

$$\vec{V} = (A^T A)^{-1} A^T (-b)$$



$$\begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} \sum_{i=1}^n I_{xi}^2 & \sum_{i=1}^n I_{xi} I_{yi} \\ \sum_{i=1}^n I_{xi} I_{yi} & \sum_{i=1}^n I_{yi}^2 \end{bmatrix}^{-1} \begin{bmatrix} -\sum_{i=1}^n I_{xi} I_{ti} \\ -\sum_{i=1}^n I_{yi} I_{ti} \end{bmatrix}$$

u 、 v 为 x 方向和 y 方向的移动速度；
 I_x 、 I_y 、 I_t 为亮度在三个轴上的偏导（也就是梯度）

Horn-Schunck (HS) 算法

算法将整个图像视为一个连续的运动场，适用于相对较平滑的场景。

- **光流场平滑假设**：场景中属于同一物体的像素形成光流场向量应当十分平滑，只有在物体边界的地方才会出现光流的突变，但这只占图像的一小部分。于是总体来看图像的光流场应当是平滑的（**稠密光流**）。
- 算法构造了一个能量函数，求光流场的问题转化为求能量函数的最小值。

$$E(u, v) = \iint [(I_x u + I_y v + I_t)^2 + \alpha^2 (\|\nabla u\|^2 + \|\nabla v\|^2)] dx dy$$

亮度不变

光流场平滑

这是一个泛函的极值问题，可以用欧拉-拉格朗日方程求解：

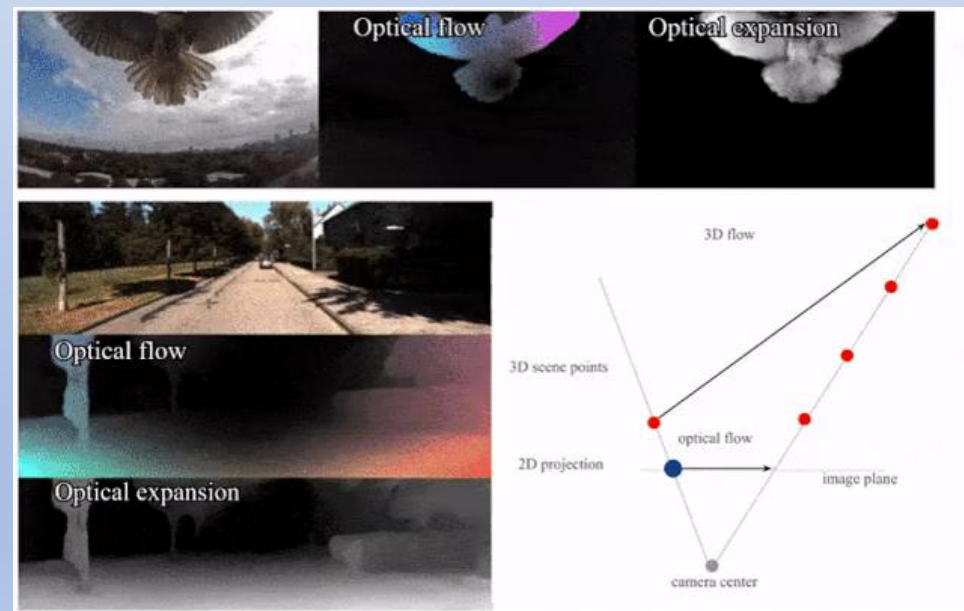
$$\begin{aligned}(I_x^2 + \alpha^2)u + I_x I_y v &= \alpha^2 \bar{u} - I_x I_t \\ I_x I_y u + (I_y^2 + \alpha^2)v &= \alpha^2 \bar{v} - I_y I_t\end{aligned}$$

HS：稠密、全局
LK：稀疏、局部

光流方法趋势

- 总而言之，光流法可以加速基于特征点的视觉里程计算法，避免计算和匹配描述子的过程，但要求相机运动较慢（或采集频率较高）。

- 现有各种光流估计方法基本上可以统一在一个框架之中，这个框架将光流信息分成两类：**保持信息和邻域信息**，光流场的恢复通过两种信息的提取和融合来实现。
- 基于**深度学习的光流估计**是目前的热点，例如FlowNet2算法可以实时取得state-of-the-art的精度。
- 随着生物视觉研究的不断深入，神经方法无疑会不断完善，也许光流计算乃至计算机视觉的根本出路就在于**神经机制的引入**。



CVPR2020: Optical Expansion将光流升级到3D场景流运动估计、目标检测、语义分割。。。

直接法运动估计

- 直接法和光流相似，也是基于灰度不变假设
- 最小化灰度差 (Photometric Error)

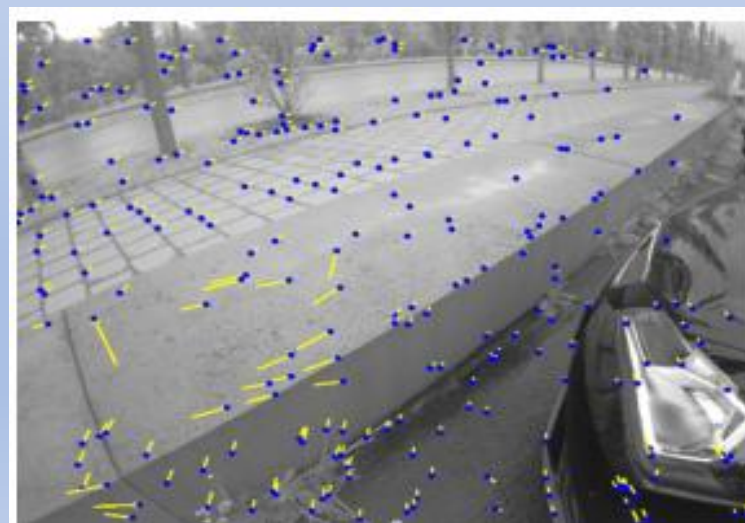
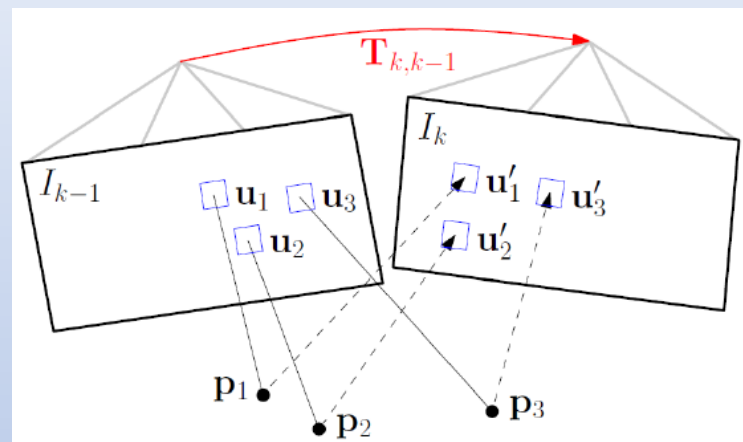
$$e = I_1(p_1) - I_2(p_2) = I_1(p_1) - I_2\left(\frac{1}{Z}K(RP + t)\right)$$

优点：

- 1) 省去计算特征点、描述子的时间
- 2) 有梯度即可，能够在特征缺失的场合下使用
- 3) 可以构建半稠密乃至稠密的地图

缺点：

- 1) 灰度不变是很理想的假设，和光流一样容易受到光照、材质、成像等因素影响
- 2) 方法依靠梯度搜索，而图像是强烈非凸的函数，使得算法容易陷入局部最优
- 3) 单个像素的信息量太贫瘠，只能统计大量像素并折中（降低精度）。



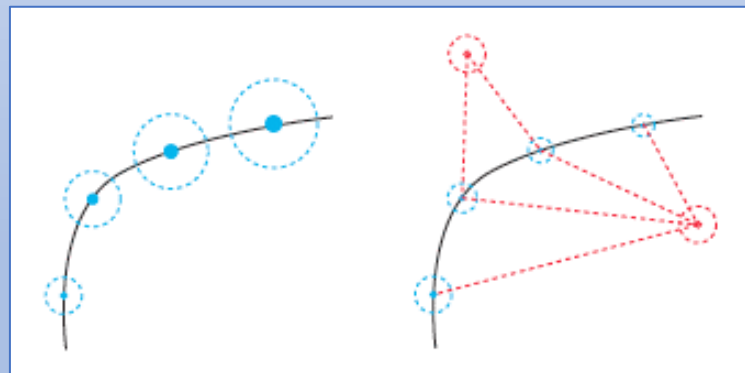
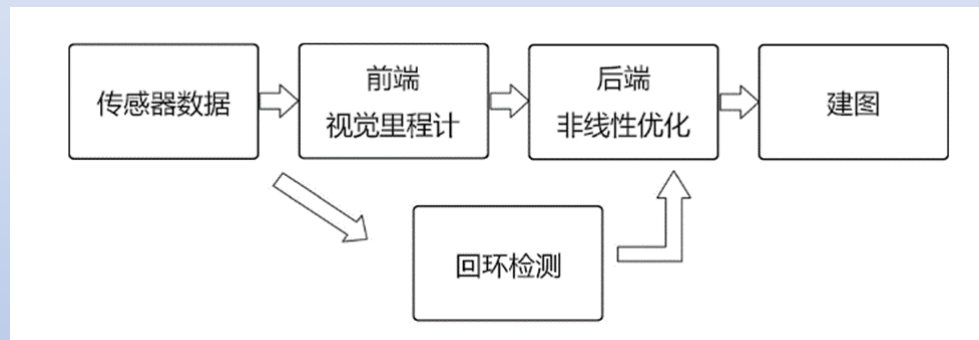
后端优化

后端优化：将相机位姿和特征点的空间位置进行同时优化，获得优化后的结果

属于状态估计问题：

从带噪声的前端数据估计内在状态

Estimated the inner state from noisy data



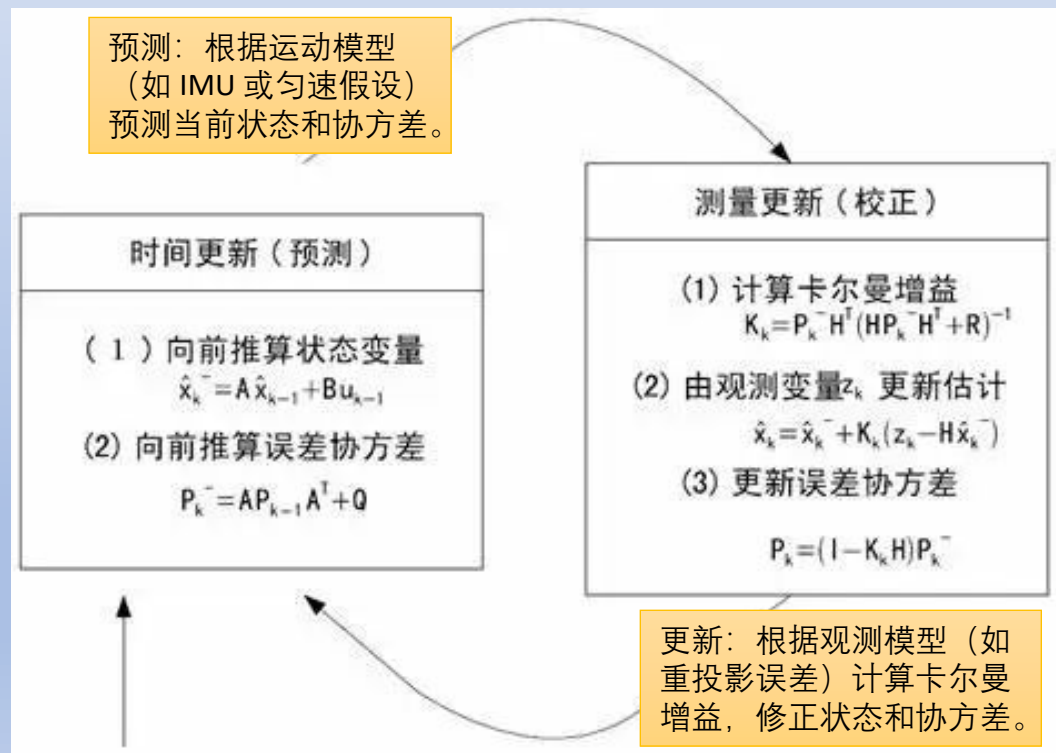
假设你闭着眼睛走到南一楼...

渐进式：有个人不断提醒你往左或者往右

批量式：过段时间睁眼看一下

渐进式：滤波方法

- 由于SLAM 本质上是一个**状态估计问题**，该问题可以归结为一个**运动方程**和一个**观测方程**，顺理成章地把 SLAM 融入到滤波框架中。早期的 SLAM 研究基本都是在滤波器的框架下。



$$\begin{cases} x_k = f(x_{k-1}, u_k) + w_k \\ z_{k,j} = h(y_j, x_k) + v_{k,j} \end{cases}$$

运动方程

观测方程

- 在线性模型、高斯状态分布下，我们将得到**卡尔曼滤波器KF**
- 在非线形模型、高斯状态分布下，可以在工作点附近线性展开，得到**扩展卡尔曼滤波器EKF**

滤波方法总结

➤ 优点:

推导简单清楚，适用各种传感器形式；易于做多传感器融合

➤ 不足:

- 1) 比如EKF方法需要把路标放进状态，由于VSLAM中路标数量很大，而且存储的状态量呈平方增长（协方差矩阵），所以难以适合大场景。
- 2、滤波方法假设马尔可夫性，假设当前状态只与上一时刻相关，而与之前状态和观测都无关，因此滤波器很难处理回环问题。
- 3、线性化误差，结果可能会发散（要求数据不能有outlier）

随着SLAM问题研究的深入及其应用逐步从小场景转向大场景，基于滤波的SLAM方法越来越受到局限。

而基于非线性优化方法倾向于使用所有的历史数据，理论上来说，能获得更好的效果。

批量式：BA非线性优化

在 **VSLAM** 中，如果不考虑运动方程，只考虑所有的观测值，整体的代价函数为：

$$\sum_{i=1}^m \sum_{j=1}^n \|e_{ij}\|^2 = \sum_{i=1}^m \sum_{j=1}^n \|z_{ij} - h(\xi_i, p_j)\|^2$$


实际观测值 观测方程

光束平差：bundle adjustment (BA)：对整体代价函数采用最小二乘求解，相当于对所有相机位姿和路标同时调整，使目标函数最小。

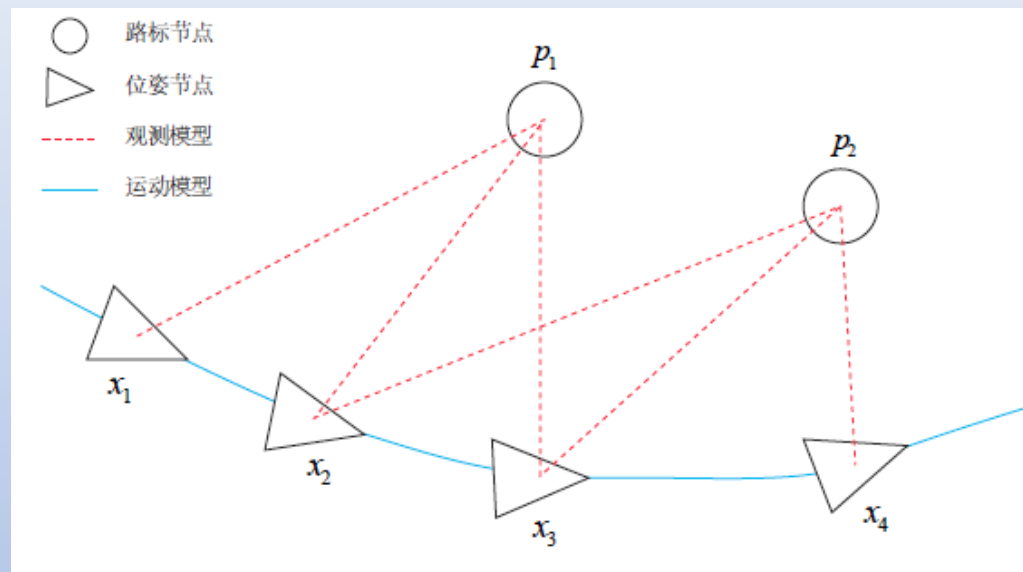
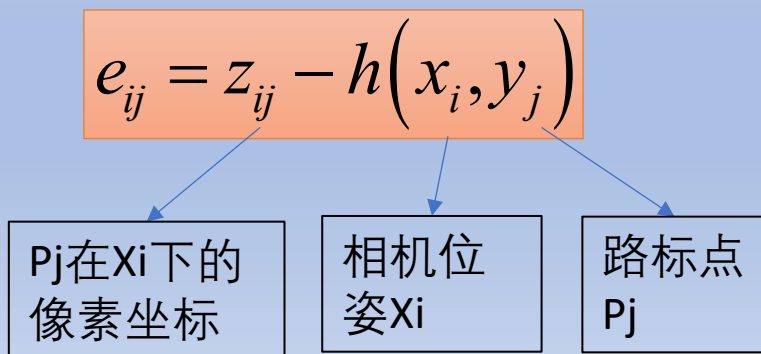
问题：研究者普遍认为非线性优化方法计算量非常大，不适合实时计算；直到最近十几年，**BA** 的稀疏特性才逐渐被认识到，才使它能够在实时的场景中应用。

BA与图优化

- BA虽是个纯优化问题，但亦可以用图模型表述出来，采用图优化方法：

- 优化变量：节点
- 误差项：边
- 问题可以用一个图来描述
- 给定初值后，迭代更新

- 在位姿 i 处对路标 j 的一次观测 z_{ij} ：



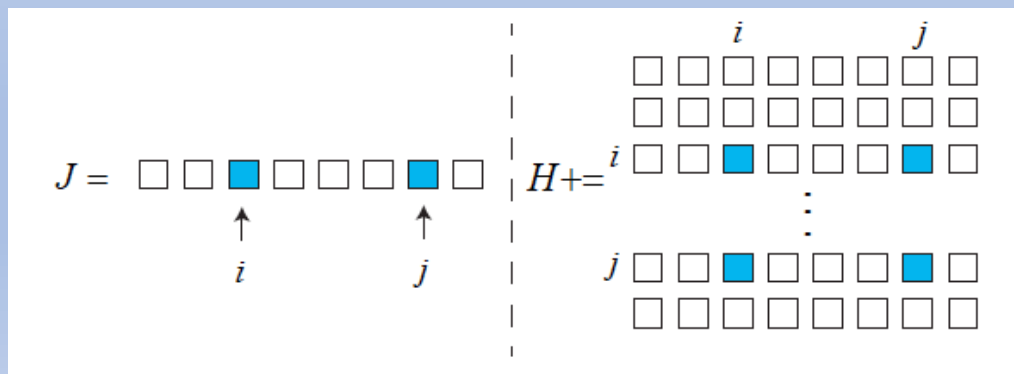
特点：

- 每个观测只关系两个变量，一个是相机，一个是路标
- 视觉BA中，不存在相机-相机，路标-路标之间的关联
- 整个误差函数由许多个这样小的项组成

BA与图优化

- 根据Gauss-Newton或Levernburg-Marquardt, 最终都需要求解 $Hx=-b$ 形式的方程, 得到迭代方向。
- 每一个误差项会对整体线程方程产生贡献: $H = \sum_{i,j} J_{ij}^T J_{ij}$
- 而当前误差仅和 i,j 相关, 导致它的雅可比为稀疏矩阵:

$$J_{ij}(x) = \left(\mathbf{0}_{2 \times 6}, \dots, \mathbf{0}_{2 \times 6}, \frac{\partial e_{ij}}{\partial \xi_i}, \mathbf{0}_{2 \times 6}, \dots, \mathbf{0}_{2 \times 3}, \dots, \mathbf{0}_{2 \times 3}, \frac{\partial e_{ij}}{\partial p_j}, \mathbf{0}_{2 \times 3}, \dots, \mathbf{0}_{2 \times 3} \right).$$



Hessian矩阵具有稀疏性

$$J = \begin{pmatrix} \frac{\partial y_1}{\partial x_1} & \frac{\partial y_1}{\partial x_2} & \frac{\partial y_1}{\partial x_3} & \cdots & \frac{\partial y_1}{\partial x_n} \\ \frac{\partial y_2}{\partial x_1} & \frac{\partial y_2}{\partial x_2} & \frac{\partial y_2}{\partial x_3} & \cdots & \frac{\partial y_2}{\partial x_n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \frac{\partial y_m}{\partial x_1} & \frac{\partial y_m}{\partial x_2} & \frac{\partial y_m}{\partial x_3} & \cdots & \frac{\partial y_m}{\partial x_n} \end{pmatrix}$$

Jacobian矩阵
一阶导数, 相当于梯度

$$H = \begin{pmatrix} \frac{\partial^2 f}{\partial x_1 \partial x_1} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_n \partial x_n} \end{pmatrix}$$

Hessian矩阵
二阶导数, 相当于曲率

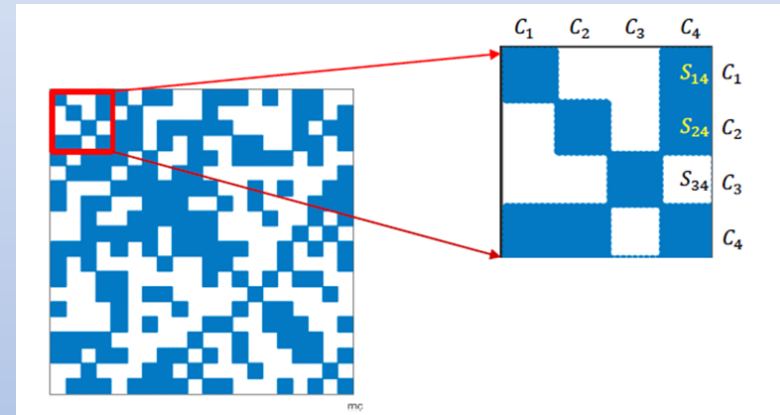
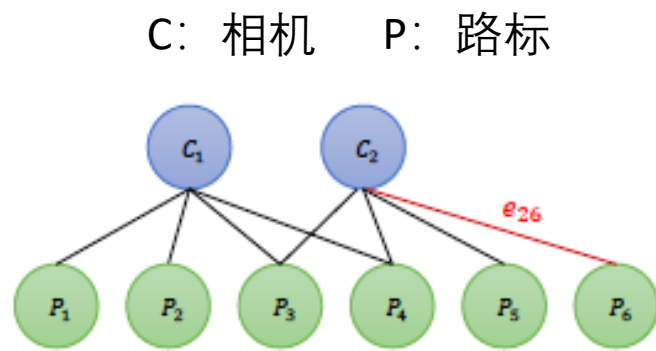
BA与图优化

图模型中存在边 $\rightarrow$ H相应地方出现非零块

思想：将所有相机位姿在前，路标在后，那么矩阵H具有一定的特殊结构

$$J = \begin{bmatrix} J_{11} \\ J_{12} \\ J_{13} \\ J_{14} \\ J_{23} \\ J_{24} \\ J_{25} \\ J_{26} \end{bmatrix} = \begin{bmatrix} C_1 & C_2 & P_1 & P_2 & P_3 & P_4 & P_5 & P_6 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \end{bmatrix}$$

<https://blog.csdn.net/zkk9527>



$$H = J^T J = \begin{bmatrix} \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \end{bmatrix}$$

<https://blog.csdn.net/zkk9527>

左上角的非稀疏矩阵反映了相机之间的共视关系（Co-visibility）。

例如：S₁₄非零，表示1,4相机看到了同一个路标点

图优化

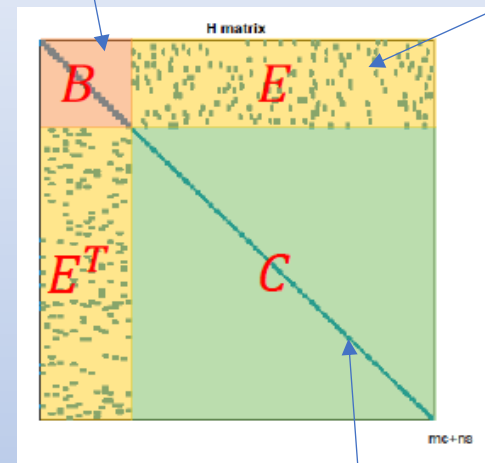
- 由于实际应用中，路标P数量远大于位姿C数量
- 利用H的特殊结构，可以大幅加速方程的求解
- 该加速手段又称边缘化方法（Marginalization）

$$\begin{bmatrix} B & E \\ E^T & C \end{bmatrix} \begin{bmatrix} \Delta x_c \\ \Delta x_p \end{bmatrix} = \begin{bmatrix} v \\ w \end{bmatrix}.$$

B: 小, 对角块;
C, 大, 对角块
E和E^T: 稠密

位姿部分 (B): 对角

观测部分 (E): 非零块



路标部分 (C): (对角)

E是非对角矩阵，不好求逆，所以先对E进行高斯消元：

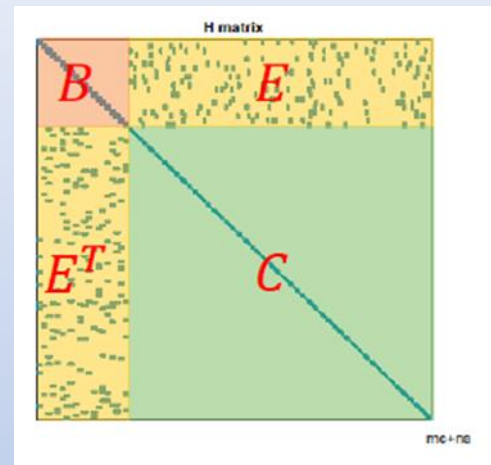
$$\begin{bmatrix} I & -EC^{-1} \\ 0 & I \end{bmatrix} \begin{bmatrix} B & E \\ E^T & C \end{bmatrix} \begin{bmatrix} \Delta x_c \\ \Delta x_p \end{bmatrix} = \begin{bmatrix} I & -EC^{-1} \\ 0 & I \end{bmatrix} \begin{bmatrix} v \\ w \end{bmatrix}.$$



$$\begin{bmatrix} B - EC^{-1}E^T & 0 \\ E^T & C \end{bmatrix} \begin{bmatrix} \Delta x_c \\ \Delta x_p \end{bmatrix} = \begin{bmatrix} v - EC^{-1}w \\ w \end{bmatrix}.$$

BA与图优化

$$\begin{bmatrix} B - EC^{-1}E^T & 0 \\ E^T & C \end{bmatrix} \begin{bmatrix} \Delta x_c \\ \Delta x_p \end{bmatrix} = \begin{bmatrix} v - EC^{-1}w \\ w \end{bmatrix}.$$



- 该方程组分为两步来求：

1. 求解上半部分，规模较小（维度和B一样），得到 Δx_c

$$[B - EC^{-1}E^T] \Delta x_c = v - EC^{-1}w.$$

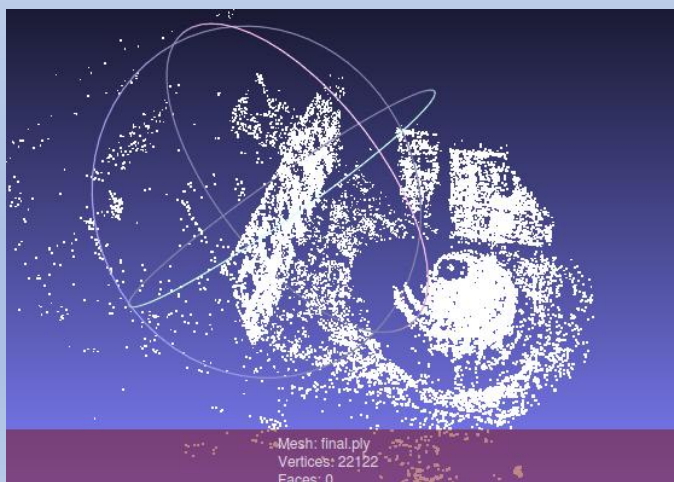
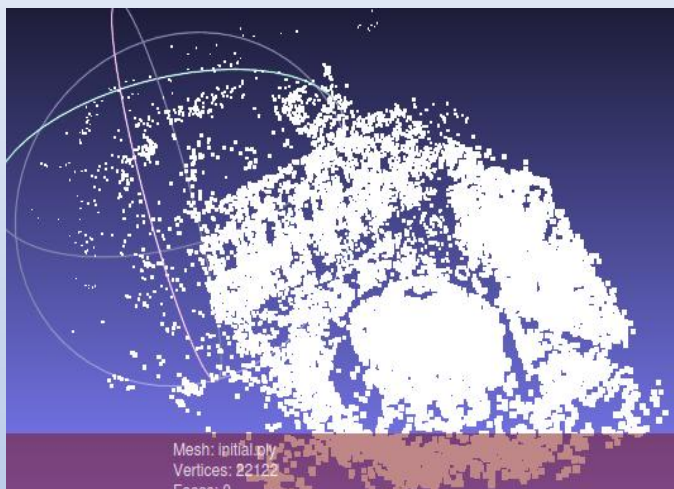
2. 将结果代入下半部分，得到 Δx_p

这个消元加速算法的**本质**是把所有的路标信息边缘化到了相机的信息中。

后端优化总结

算法	缺点	优点
KF/EKF	假设噪声为高斯分布，在高维状态空间时计算效率较低，不适合大规模场景的地图构建	原理简单，小场景下收敛性好。
PF	会不可避免出现粒子退化现象，每个粒子需包含整张地图信息导致不适合大场景地图构建	将运动方程表示为一组粒子，通过蒙特卡罗方法求解，能够处理非线性非高斯噪声。
EM	计算量大，不能用于大规模场景	有效解决了数据关联问题
图优化 Pose Graph	对闭环检测算法的要求严格	出现多种图优化框架，能够有效解决滤波器算法的缺陷，能用于大规模场景的地图创建

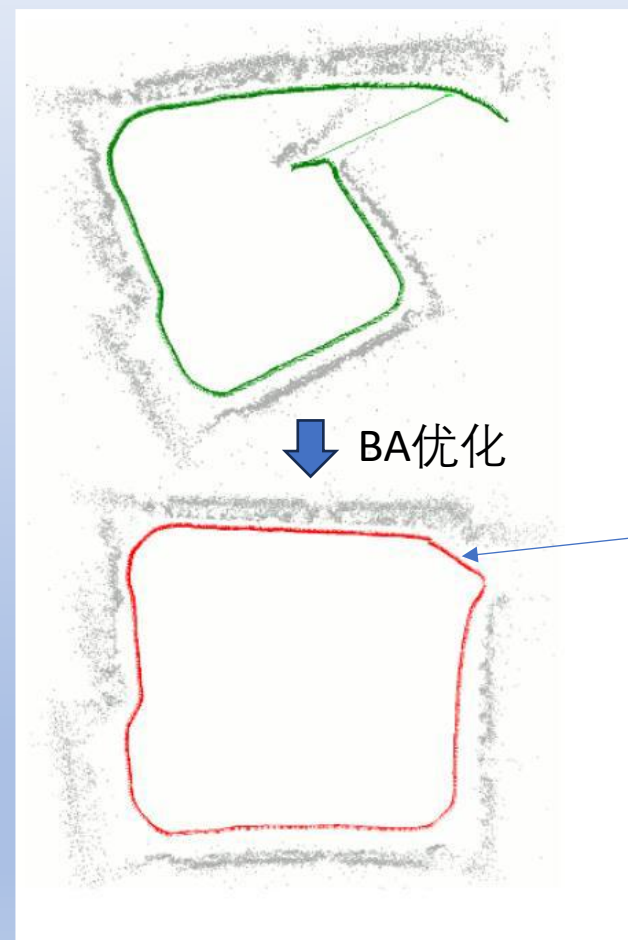
效果和问题



BA优化前后

为消除累计误差，获得全局一致的地图和轨迹，仅仅通过帧间数据匹配估计运动位姿还是远远不够的。

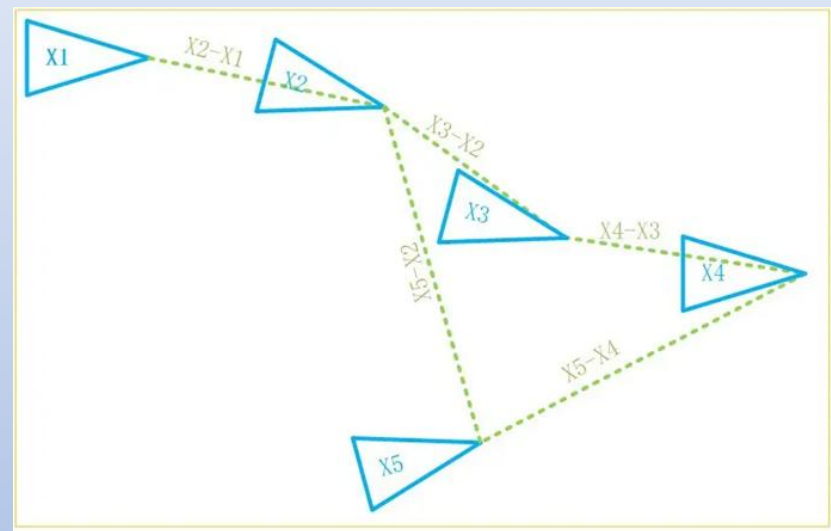
需要在全局数据中进行处理才能得到更好的结果。



残留问题

回环检测 Loop Closing

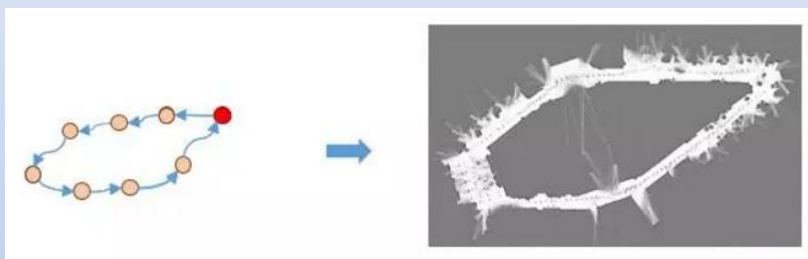
- 回环检测的必要性
 - VO和后端都**存在误差**
 - SLAM建图与定位是耦合的：**误差会累计**
 - 回环检测保证了轨迹与地图的**全局一致性** (Globally Consistent)
- Loop Closing的步骤
 1. 检测到回环的发生
 2. 计算回环修正帧与当前帧的运动
 3. 验证回环是否成立
 4. 闭环



如果当前帧与之前的某一帧（不是相邻帧）建立位姿约束关系就叫做**回环**。回环起到了减小累积误差的作用。

回环检测

• 基于关键帧匹配的方法 (matching based)



• 基于里程计的方法 (Odometry based)

• 基于外观的方法 (Appearance based)

- 外观方法是主流，核心在于衡量图像间的相似性

并不容易

表 12-1 回环检测的结果分类

算法 \ 事实	是回环	不是回环
是回环	真阳性 (True Positive)	假阳性 (False Positive)
不是回环	假阴性 (False Negative)	真阴性 (True Negative)

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP}), \quad \text{Recall} = \text{TP} / (\text{TP} + \text{FN}). \quad (12.2)$$



False Positive

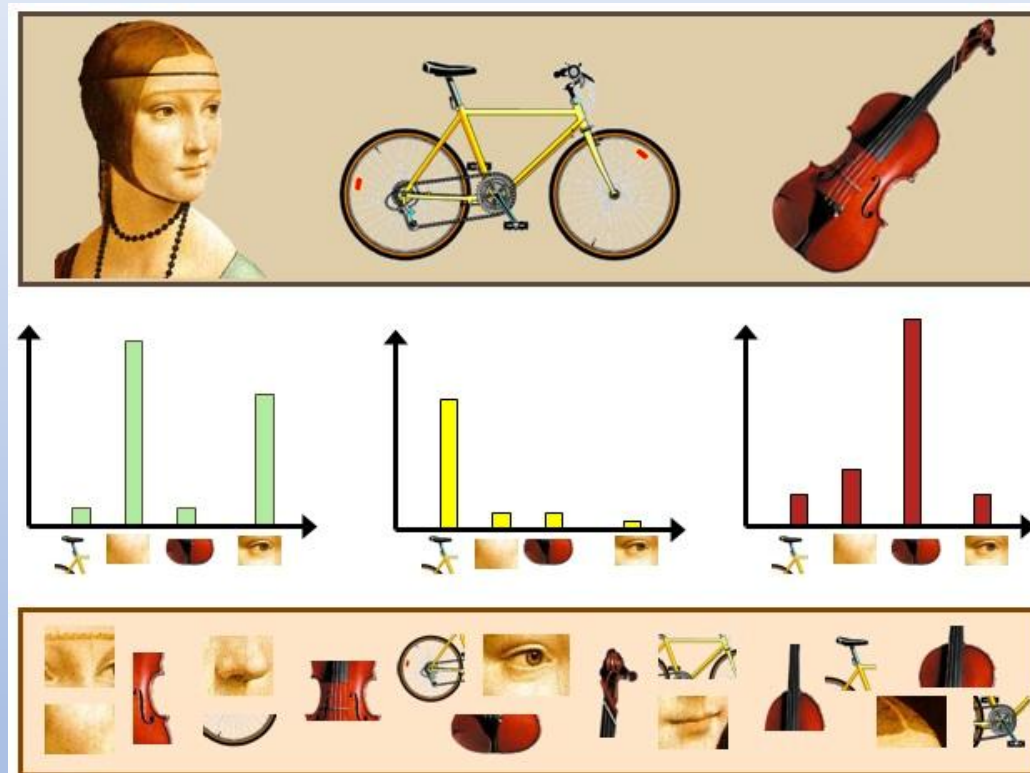
False Negative

相似的两个走廊

不同光照的同一个位置

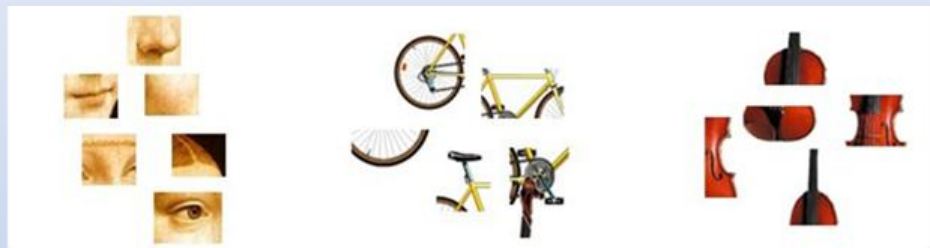
词袋方法

- Appearance-based主要方法词袋
Bag-of-Words
- BoW是对特征的聚类
 - 特征聚类形成了Word
 - 许多Word组成了Dictionary
 - 图像的相似性=Word的相似性

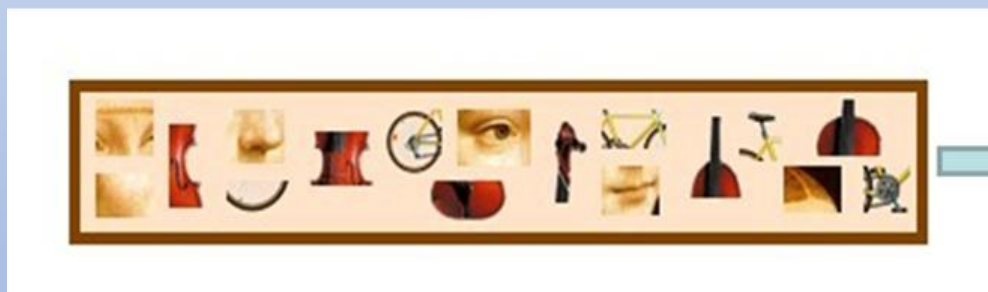


只看Word的有无，无视Word的顺序

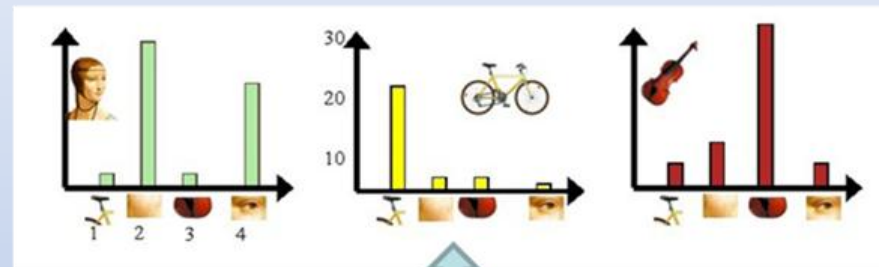
BOW框图



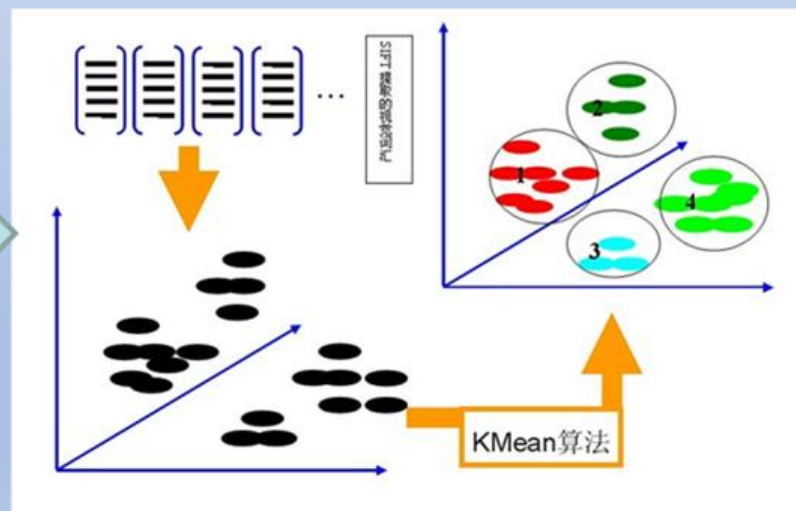
图像分块



局部特征描述（例如SIFT）



用视觉单词直
方图表示图像，
并计算图像相
似度



聚类中心定义
为视觉单词。

所有视觉单词
组成视觉词典

所有局部特征进行聚类

相似度计算

- 原则上比较Word即可计算图像相似度。但一些Word很常见，另一些则很罕见，他们权重一样吗？
- 采用TF-IDF权重 η :

$$\text{词频(TF)} = \frac{\text{某个词在文章中的出现次数}}{\text{文章的总词数}}$$

$$\text{逆文档频率(IDF)} = \log\left(\frac{\text{语料库的文档总数}}{\text{包含该词的文档数} + 1}\right)$$

$$\text{TF-IDF} = \text{词频(TF)} \times \text{逆文档频率(IDF)}$$

TF: 该单词在本图像中频率越高，则重要性越高。

IDF: 该单词在其他图像中出现的频率越低，则信息量越大。

- 相似度:

加权词袋

$$A = \{(w_1, \eta_1), (w_2, \eta_2), \dots, (w_N, \eta_N)\} \triangleq \mathbf{v}_A.$$

L1相似度

$$s(\mathbf{v}_A - \mathbf{v}_B) = 2 \sum_{i=1}^N |\mathbf{v}_{Ai}| + |\mathbf{v}_{Bi}| - |\mathbf{v}_{Ai} - \mathbf{v}_{Bi}|.$$

回环检测

- **1. 回环验证（几何一致性检验）**

- 因此，回环检测成功后，需要先进行几何验证：特征匹配：几何模型拟合：重投影检查。

- **2. 添加回环约束**

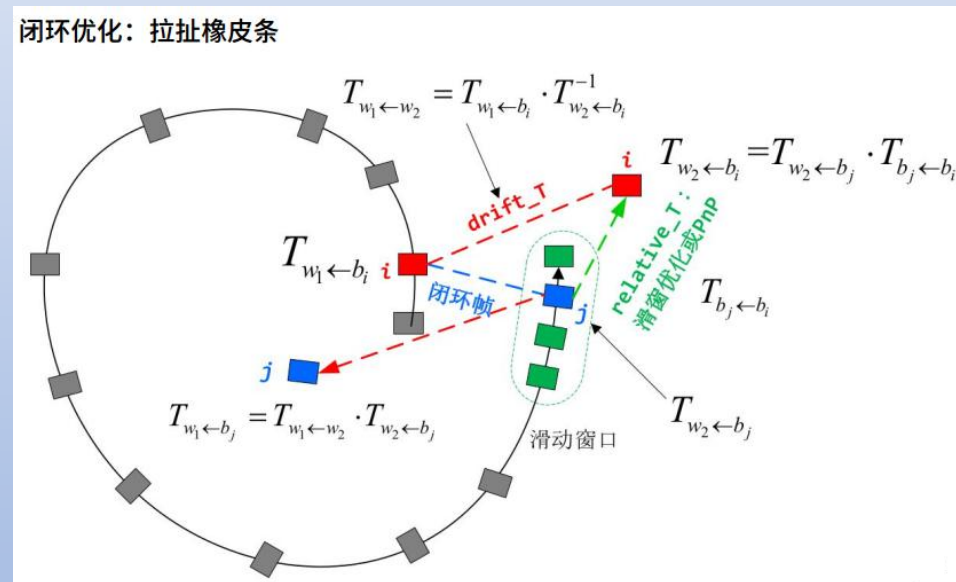
验证通过后，系统将回环关系作为强约束加入后端优化中：

- **3. 位姿图优化（Pose Graph Optimization）**

- 为高效修正漂移，大多数 SLAM 系统会执行全局位姿图优化：优化对象：仅优化所有关键帧的位姿（3 个平移 + 3 个旋转），地图点暂时固定。

- **4. 全局地图更新（地图融合与一致性）**

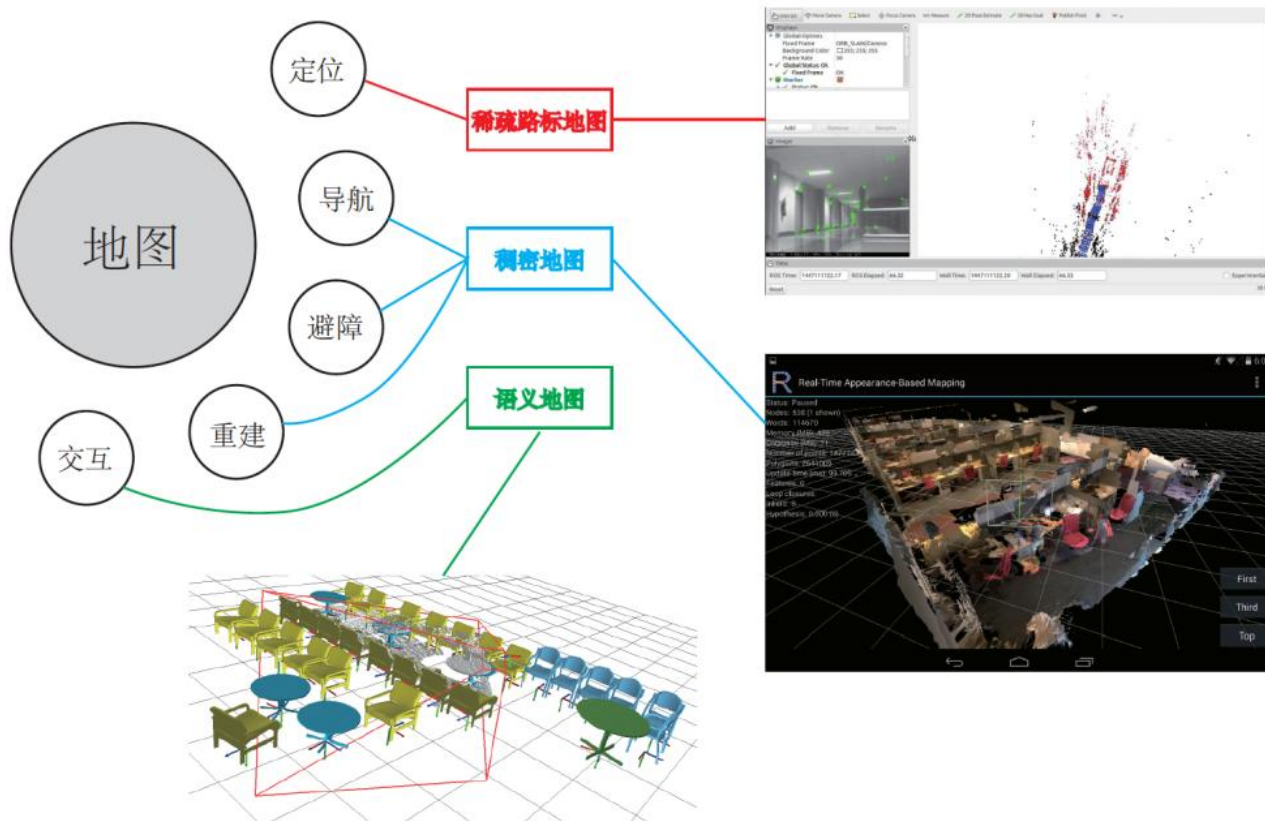
- 位姿图优化后，地图点需要根据新的关键帧位姿进行变换，并处理重复的地图点：



建图 Mapping

建图是SLAM的两大目标之一。**SLAM中的地图：所有路标点的集合。**

但是作为一种底层技术，根据上层任务不同，有不同的建图需求。

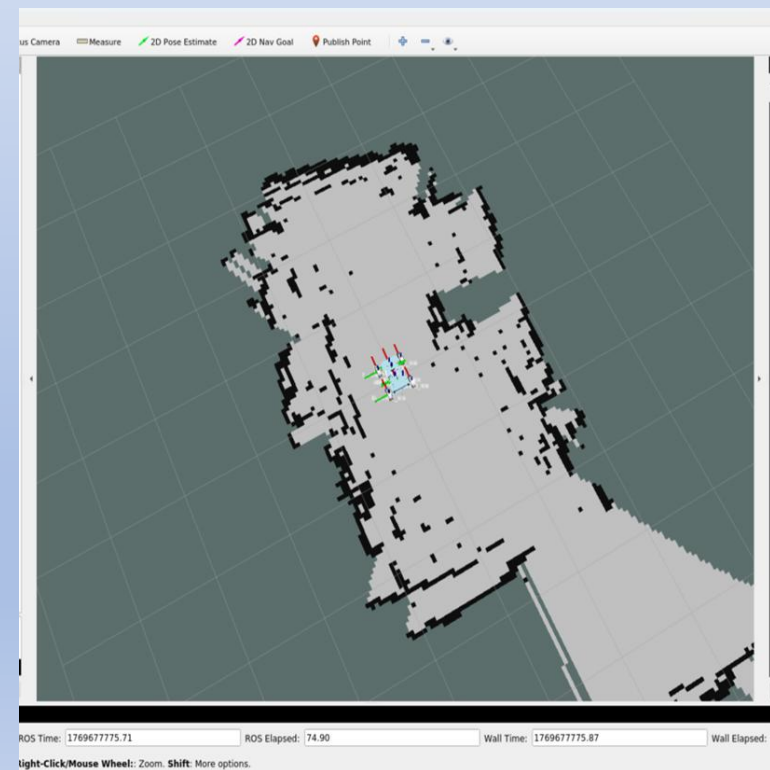
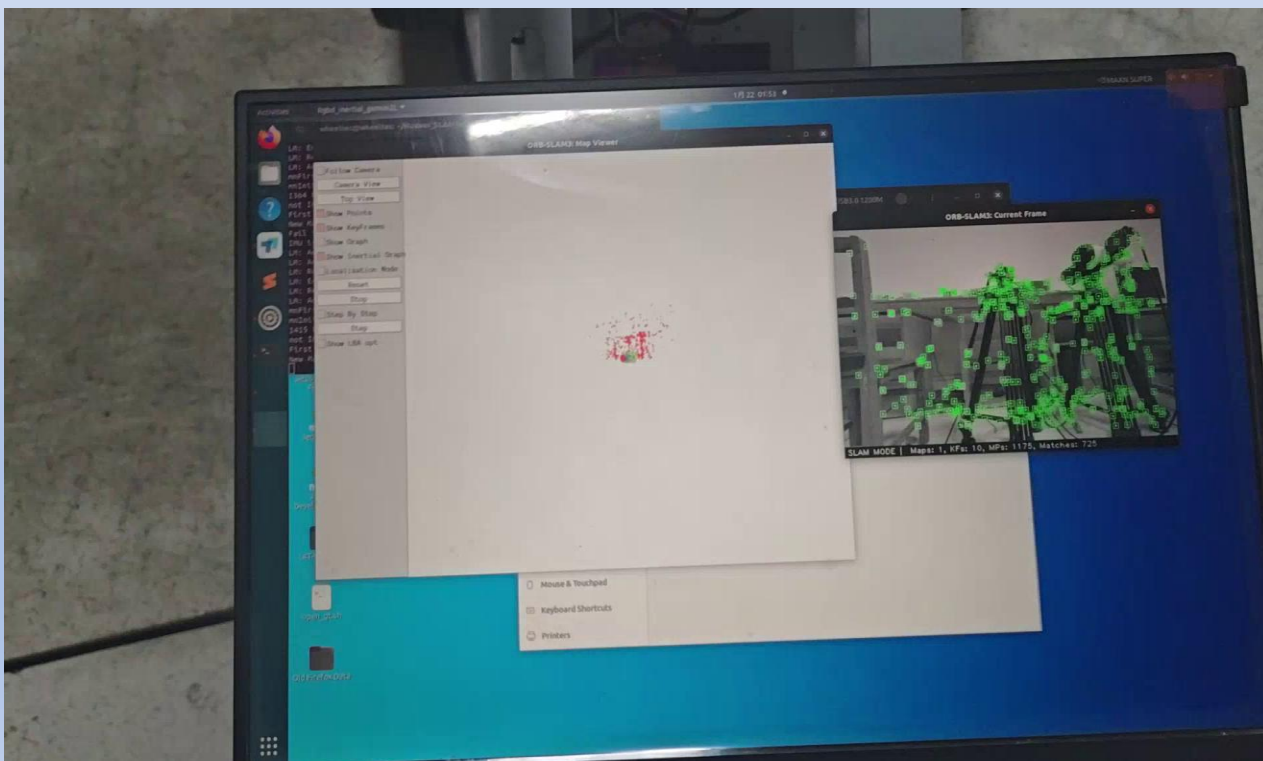


- **定位**仅需特征点地图
- **导航和避障**需要稠密障碍物信息
- **重建**需要稠密的物体表面信息
- **交互**等高层任务需要语义信息

特征点地图仅满足很小一部分需求

SLAM中的特征点地图

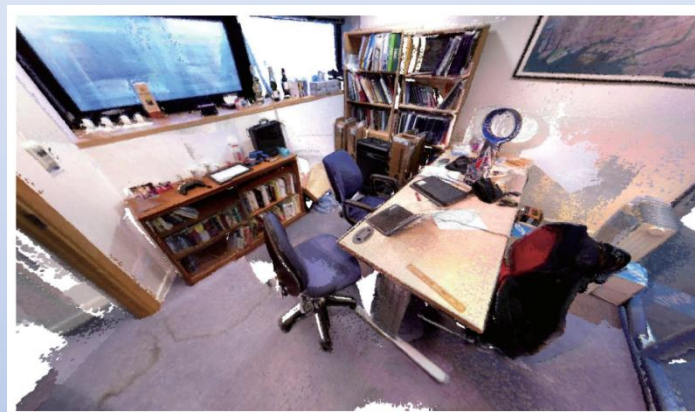
- 等价于地图点的位置估计
- 单目SLAM中可看成地图点的深度估计



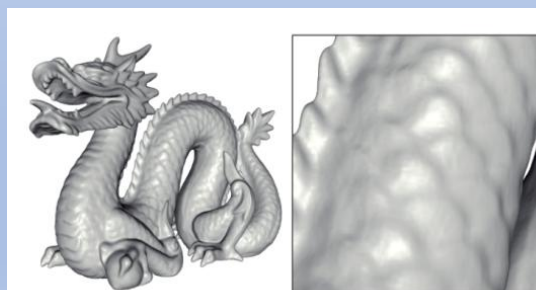
RGBD稠密建图

- 点云
- 网格/面片
- TSDF/voxel
- 八叉树

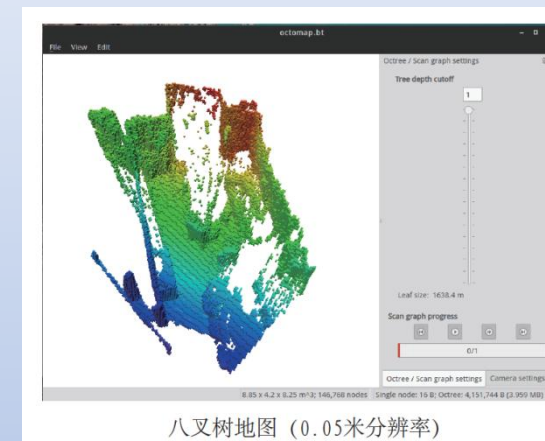
RGB-D相机完全通过传感器中硬件测量得到深度，无须消耗大量的计算资源来估计。故RGB-D传感器进行稠密建图是相对容易的。



泊松重建：从点云到网格



泊松重建示例



八叉树地图 (0.05米分辨率)

Surfel: 以面片为单位的建图



Surfel重建示例

SLAM技术展望

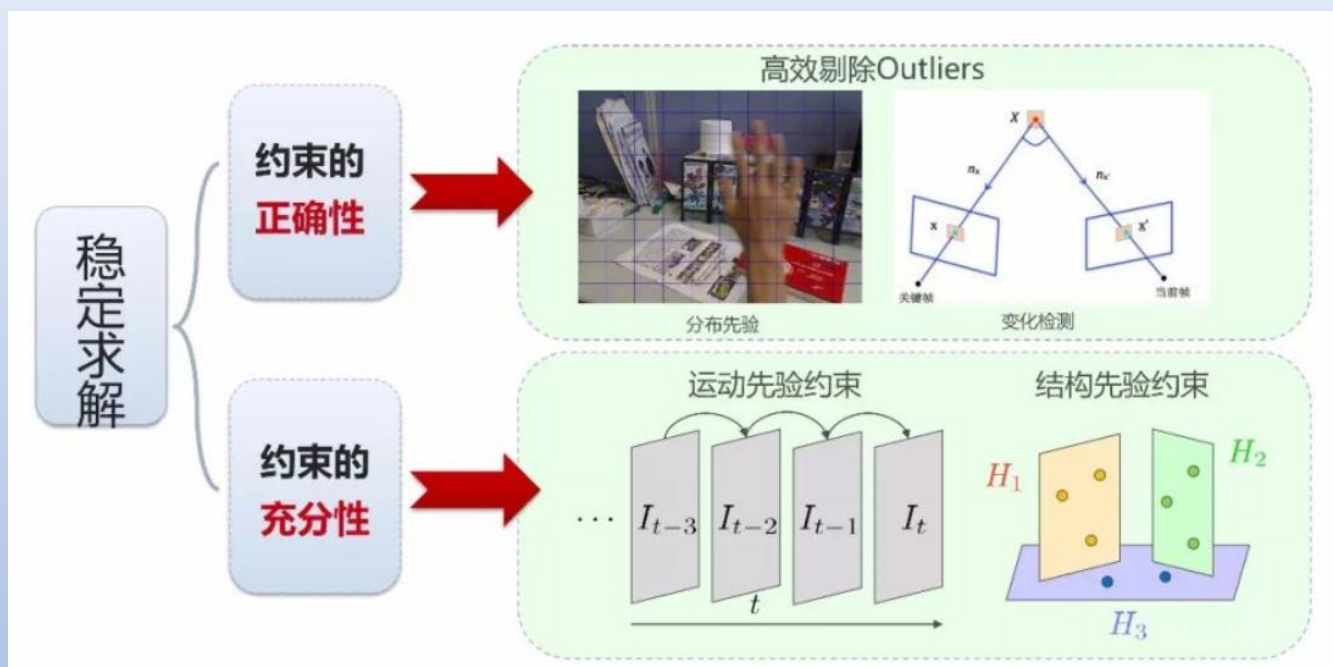
视觉SLAM面临的挑战



精度和稳定性方面，因为环境往往是动态变化的，弱纹理和重复纹理问题很容易造成视觉特征匹配不准确，从而导致优化计算不稳定。

实时性方面，主要是在大规模场景下，SLAM计算复杂度高，而低功耗设备的计算能力有限，如何做到实时计算。

关键问题1-约束条件



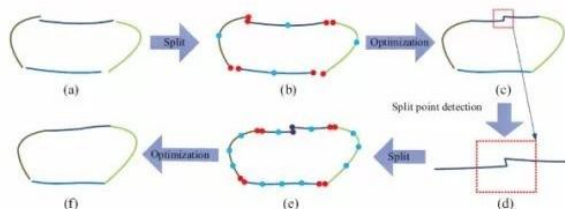
SLAM本质上就是一个目标函数构造和优化问题。

- 1) 要保证目标函数里的约束项都是正确的，即尽可能剔除错误的匹配。
- 2) 在弱纹理情况下，匹配点过少使得约束项不足，两方面的思路：
 - a. 添加一些**运动先验**的约束，比如IMU。
 - b. 添加**场景的先验结构**约束，比如一些平面结构约束。

关键问题2-效率

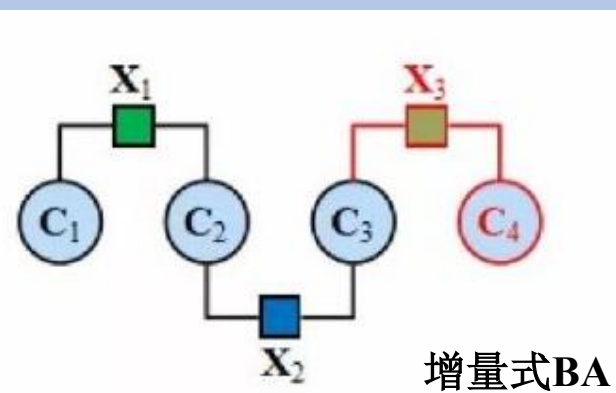
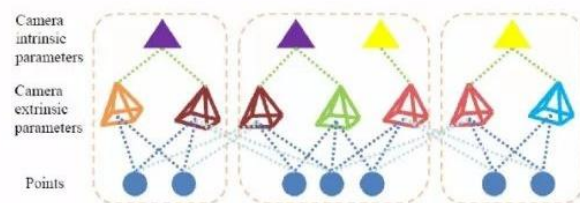
基于分段的BA

- 将长序列切分为若干短序列
- 由粗到细层次优化



分布式BA

- 切分子问题，各子问题独立优化
- 处理不同子问题共享变量



增量式BA

	C ₁	C ₂	C ₃	C ₄
C ₁	■	■+■		
C ₂		■+■	■+(■)	
C ₃			■+(■)	(■)
C ₄				(■)

解决计算效率非常关键的环节就是BA：相机参数和三维点数目可能非常多，复杂度非常高。

即使已经利用了H矩阵的稀疏性，求解效率依然是BA的一大挑战。

展望

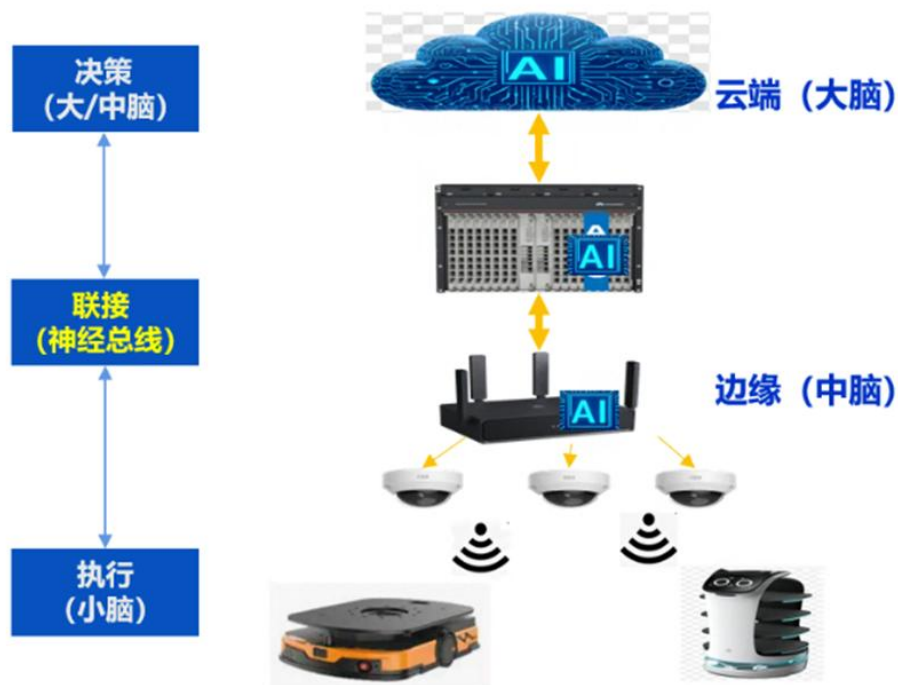
- SLAM已有众多的开源方案
 - 开源能够让用户了解其原理, 促进研究的发展。
- 今后的方向
 - 多传感器融合
 - V+IMU, V+GPS, V+ODOM
 - SLAM+深度学习
 - Depth learning
 - Stereo learning
 - Descriptor Learning
 - Semantic mapping/BA
 - Etc.

表 14-1 常用开源 SLAM 方案

方案名称	传感器形式	地址
MonoSLAM	单目	https://github.com/hanmekim/SceneLib2
PTAM	单目	http://www.robots.ox.ac.uk/~gk/PTAM/
ORB-SLAM	单目为主	http://webdiis.unizar.es/~raulmur/orbslam/
LSD-SLAM	单目为主	http://vision.in.tum.de/research/vslam/lsdslam
SVO	单目	https://github.com/uzh-rpg/rpg_svo
DTAM	RGB-D	https://github.com/anuranbaka/OpenDTAM
DVO	RGB-D	https://github.com/tum-vision/dvo_slam
DSO	单目	https://github.com/JakobEngel/dso
RTAB-MAP	双目/RGB-D	https://github.com/introlab/rtabmap
RGBD-SLAM-V2	RGB-D	https://github.com/felixendres/rgbdslam_v2
Elastic Fusion	RGB-D	https://github.com/mp3guy/ElasticFusion
Hector SLAM	激光	http://wiki.ros.org/hector_slam
GMapping	激光	http://wiki.ros.org/gmapping
OKVIS	多目 +IMU	https://github.com/ethz-asl/okvis
ROVIO	单目 +IMU	https://github.com/ethz-asl/rovio

难题：基于分布式算力的大小脑协同实现具身监控规划导航一体化

- 复杂动态环境的视觉导航仍然难以处理弱纹理、遮挡、环境变化等问题；
- 上帝视角AI Sensor未充分挖掘其在定位、导航的应用潜力，与具身本体形成了信息孤岛；
- 多视角AI Sensor分辨率、视场角、时延差异，在时间、空间上需要对齐；
- 现有具身智能推理决策依赖高算力&高能耗实现复杂操作，需要按需切分、向上卸载；
- 传统集中式推理→控制架构存在延迟高、带宽不足问题。



技术目标

- **超低时延协同架构：**基于边端协同的大小脑分布式推理框架，系统整体执行频率 $>10\text{Hz}$ ，端到端控制时延 $<100\text{ms}$ ；
- **高精度定位：**上帝视角+本体视觉协同导航，联合定位误差 $<2\text{cm}$ ；
- **安全检测与避障：**联合监控，实现行人、桌椅、货物识别率 $>95\%$ ，实现定位和避障；
- **动态路线规划：**对视野内的机器人/行人移动进行预测，实现0碰撞0冲突的路径规划；
- **自主脱困机制：**定位导航失效时，能够本体自主脱困，在确保安全无碰撞的情况下找回定位，继续导航。

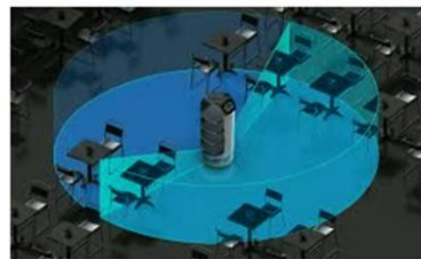
关键技术

1、自动化多相机位姿配准



基于移动机器人的相机外参标定

2、动态场景长期建图与定位



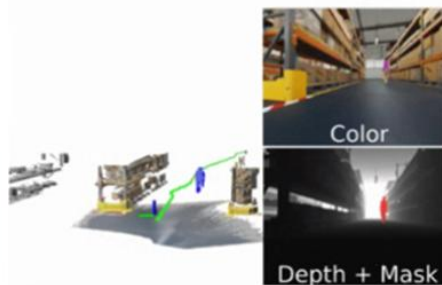
基于多视图融合的Life-long SLAM

3、多目标3D检测与识别

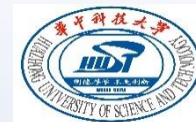


静态/动态物体 (桌椅货架、行人)
3D检测与识别

4、多视角协同的路径规划与避障



动态路径规划与避障



華中科技大學

明德厚学、求是创新

马杰